EAASP - 企业自主智能体支撑平台设计规范 v1.7

1. 执行摘要
2. 总体架构
    2.1 四层模型
    2.2 三项跨层机制
    2.3 设计原则
3. L4 — 人机协作层（Human engagement layer）
    3.1 体验平面
    3.2 集成平面
    3.3 控制平面
    3.4 持久化平面
    3.5 多租户与组织拓扑
    3.6 安全架构
    3.7 L4 的其他能力
4. L3 — 智能体治理层（Agent Governance Layer）
    4.1 策略引擎
    4.2 审批闸门
    4.3 审计服务
    4.4 MCP 注册表
    4.5 Hook 作用域层级
    4.6 四类 hook handler 类型
    4.7 Hook 生命周期事件
5. L2 — 技能资产层（Skill assets layer）
    5.1 层级架构
    5.2 Skill 分类（Skill taxonomy）
    5.3 Skill 内容结构
    5.4 Skill 中的作用域 hooks
    5.5 运行时亲和性
    5.6 仓库 MCP 工具
    5.7 晋升流水线（Promotion pipeline）
    5.8 L2 与相邻层的接口
6. L1 内部抽象机制
    6.1 运行时选择器 (Runtime Selector)
    6.2 适配器注册表 (Adapter registry)
    6.3 Hook 桥接器 (Hook bridge)
    6.4 遥测采集器 (Telemetry collector)

6.5 运行时接口契约
7. L1 — 智能体执行层 (Agent excution layer)
7.1 运行时分层（Runtime tiers）
7.2 运行时池中的生命周期
7.3 单实例内部组成
8. L3/L4 接口：五个 API 契约
8.1 契约 1：策略部署
8.2 契约 2：意图网关
8.3 契约 3：技能生命周期
8.4 契约 4：遥测采集（Telemetry ingest）
8.5 契约 5：会话控制（Session control）
8.6 会话创建：三方握手（Three-way handshake）
9. L1/L3 接口：hooks 作为连接机制
9.1 L3 治理如何触达 L1
9.2 L1 如何回报给 L3
9.3 skills 如何桥接 L1 与 L3
9.4 审批闸门如何跨越 L1/L3 边界
10. Hooks：跨层强制执行架构
10.1 Hooks 起源：L4 管理控制台
10.2 Hooks 编译：L3 策略编译器
10.3 Hooks 部署：L3 托管设置
10.4 Hooks 桥接：L1 Hooks 桥接器
10.5 Hooks 执行：L1 内联触发
10.6 Hooks 遥测流：L1 → L3 → L4
10.7 Hooks 审批流：L1 → L3 → L4 → L1
10.8 Hooks 保证：拒绝优先（Deny always wins）
11. 部署拓扑
11.1 边缘 / CDN 分区
11.2 L4 人机协作集群
11.3 L3 智能体治理集群
11.4 L2 技能资产集群
11.5 L1 智能体执行区域
11.6 数据层
11.7 外部集成
11.8 网络架构
11.9 高可用与容灾恢复
12. 演进策略
阶段 1：L1 内核 + 基础 hooks（第 1–4 周）
阶段 2：L2 技能资产 + L1 抽象（第 5–12 周）
阶段 3：L3 智能体治理 + L4 基础能力（第 13–20 周）
阶段 4：完整 L4 + L3 成熟 + Tier 3（第 21–30 周）

阶段 5：生态扩展（持续进行）
13. 设计反模式
14. 完整设计决策轨迹
15. 实施团队分工
15.1 TypeScript / React 团队（10–12 名开发者）
15.2 Java 团队（6–7 名开发者）
15.3 Python 团队（6–7 名开发者）
15.4 Rust 团队（2–3 名开发者）
15.5 DevOps 团队（1–2 名开发者）
15.6 分工原则
15.7 人员配置汇总
16. 实施团队分工（Python为主）
16.1 Python 团队（5–8 名开发者）
16.2 TypeScript / React 团队（2 名开发者）
16.3 Java 团队（1–2 名开发者）
16.4 Rust 开发者（1 名）
16.5 DevOps（1 名开发者）
16.6 分配原则
16.7 人员配置汇总

1. 执行摘要

本文档是企业自主智能体支撑平台（EAASP）的权威设计规范。它定义了一个四层架构，并配套三项跨层机制，使企业能够将异构的 AI 智能体运行时以统一的平台服务形态进行部署、治理与规模化扩展，从而提升员工生产力。

四个层级自底向上编号，各自承担不同职责：L4 人机协作层面向人类与业务系统；L3 智能体治理层约束并控制智能体能做什么；L2 技能资产层管理智能体所消费的知识；L1 智能体执行层承载执行工作的异构智能体。三项跨层机制贯穿所有层：hooks（确定性强制执行）、skills（智能化知识）、MCP（通用集成协议）。

平台通过五轮迭代，逐步回答更深层的架构问题。最终设计解决了六大核心挑战：

- **运行时无关的执行：**L1 是智能体执行层，内部包含抽象机制（运行时选择器、适配器注册表、hook 桥接器、遥测采集器）。任何实现 12 个方法运行时接口契约的智能体都可加入运行时池中——从 Anthropic 的 harness 智能体（Octo、Claude Code），到社区 CLI 智能体（Aider、Goose），再到框架型智能体（LangGraph、CrewAI）。智能体按会话通过五种可配置策略进行选择。
- **智能体治理：**L3 将企业策略编译为受管 hooks，并通过 managed-settings.json 部署。L1 的 hook 桥接器确保在所有智能体上实现一致的强制执行，包括那些不原生支持 hook

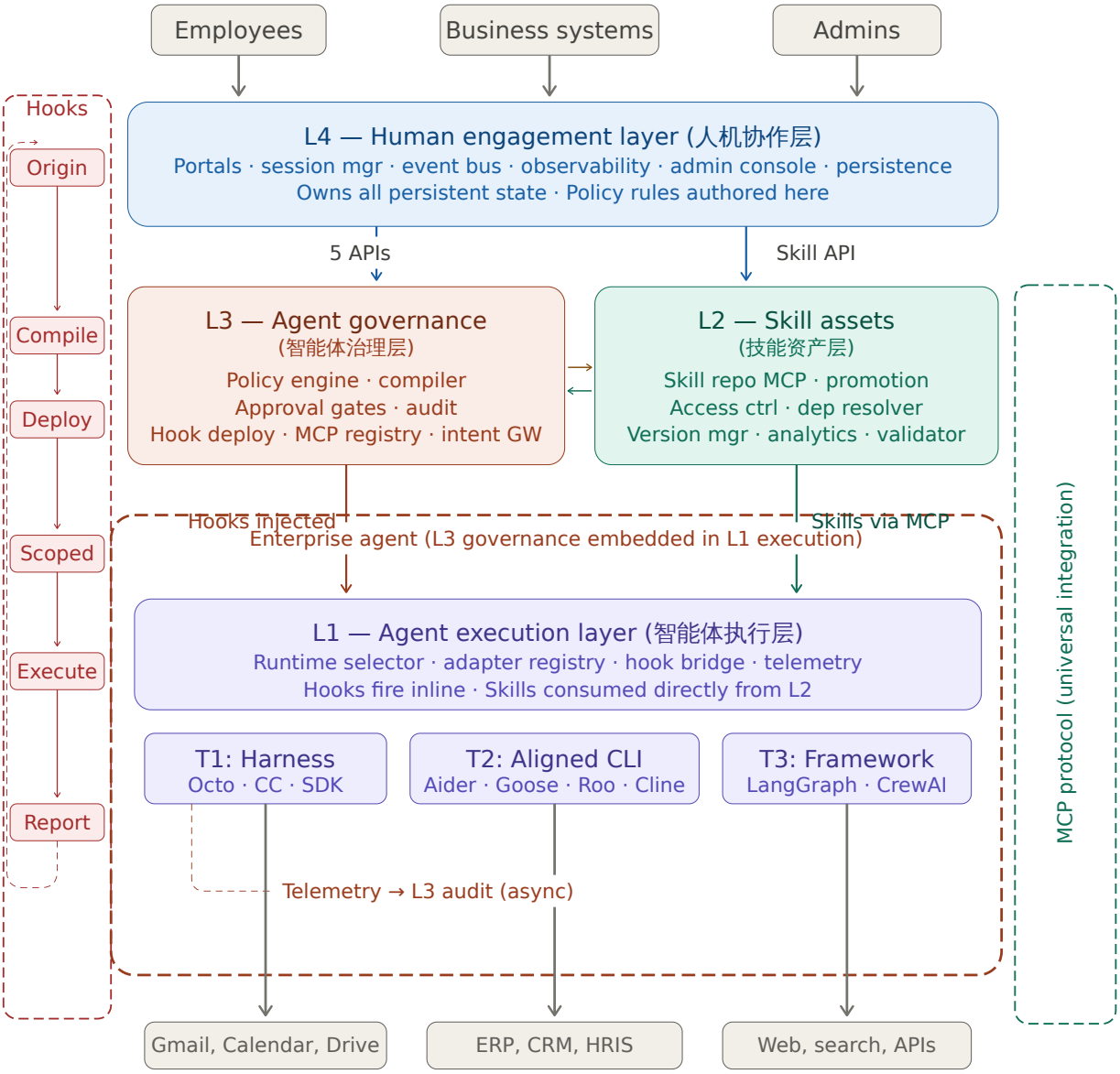
的运行时。四类 handler（command、http、prompt、agent）构成升级处理阶梯。“拒绝优先”（Deny always wins）。

- **智能工作流：**业务流程在 L2 中被编码为 workflow-skills（而非僵化的 YAML 流水线），从而保留智能体的智能性与适应性。Skill 在 YAML frontmatter 中携带作用域 hooks，以确定性方式强制执行不可妥协的质量与合规规则。智能与保障在同一份资产中共存。
- **人机协作：**L4 提供完整的人类界面，包含四个内部平面（体验、集成、控制、持久化），并定义了五个用于 L3/L4 通信的 API 契约。L4 负责员工、管理、业务事件、可观测性以及所有持久化状态。
- **技能资产管理：**L2 是完整的运营层，而不是被动的仓库。它提供六个运行时服务：Skill 仓库 MCP server、晋升引擎、访问控制、依赖解析器、版本管理器、使用分析。Skill 具备版本化能力，按组织范围进行隔离，并通过四阶段晋升流水线进行治理。
- **渐进式演进：**平台通过在四层架构对齐的五个阶段中增量添加组件来扩展既有 Octo 运行时。每个阶段都能独立交付价值，无需基础设施重建。

核心论断：平台对运行时无关。治理是统一的。智能通过 Skills 流动。Hooks 提供确定性保障。L4 服务人类。L3 治理智能体。L2 管理知识。L1 执行工作。“拒绝优先”（Deny always wins）。

2. 总体架构

该架构由自底向上编号的四个层级构成，每一层都有清晰的定位，并配有贯穿所有层级的三项跨层机制。本章在后续章节详细展开各组件之前，先给出完整的结构总览。



L4 manages L3 + L2 as peers · L2 delivers skills directly to L1 (not via L3) · L3 + L1 = enterprise agent
Hooks: L4 origin → L3 compile/deploy → L2 scoped → L1 execute → report back (full cycle)
Deny always wins · 4 scope levels · 4 handler types · 9 lifecycle events

2.1 四层模型

层级	职责	关键组件	状态归属	强制机制
L4 人机协作（Human	面向人类与业务系统：门户、会话、事	四个平面：体验、集成、控制、持久	持久化：用户、历史、成本、会话	身份认证（AuthN）、会话

engagement）	件、可观测性、管理与持久化状态	化		生命周期
L3 智能体治理（Agent governance）	智能体控制：策略、审批闸门、审计、资产生命周期、受管 hooks	策略引擎、审批闸门、审计服务、MCP 与 skill 注册表	配置：策略、RBAC、hooks、managed-settings	受管 hooks，“拒绝优先”（Deny always wins）
L2 技能资产（Skill assets）	知识与资产：版本化 skills、作用域 hooks、晋升、访问控制、分析	Skill 仓库 MCP、晋升引擎、依赖解析器、版本管理器、分析	版本化：skill 内容、元数据、访问范围、使用指标	frontmatter 中的 skill 作用域 hooks
L1 智能体运行（Agent execution）	执行：推理—规划—行动的异构智能体	内部抽象（选择器、适配器、hook 桥接、遥测）+ 运行时分层	短暂态：对话、内核状态、文件	原生 hooks 或桥接 hooks

各层通过定义明确的边界进行交互。L4 通过五个 REST API 契约与 L3 通信（策略部署、意图网关、Skill 生命周期、遥测采集、会话控制）。L3 与 L1 的通信通过在会话创建时嵌入的 hooks 实现，而不是通过逐条消息的 API 调用。L2 通过其 MCP 服务器接口被 L4（流程设计器创建 Skills）、L3（治理控制晋升）以及 L1（智能体在执行期间读取 Skills）访问。

2.2 三项跨层机制

- **Hooks：**在生命周期的每个事件上进行确定性强制执行。由 L3 部署的受管 hooks 属于企业级能力，不能被覆盖。携带在 L2 skill frontmatter 中的 skill 作用域 hooks 在工作流执行期间激活。L1 中的 hook bridge 将强制能力扩展到不原生支持 hooks 的智能体。四类 handler 构成升级处理阶梯：command（毫秒级）、http（~300ms）、prompt（~2s）、agent（~15s）。
- **Skills：**由 L2 管理的智能化知识资产，L1 中任何智能体都可消费。Workflow-skills 将业务流程编码为带作用域 hooks 的自然语言作战手册。Production skills 编码输出类型的最佳实践。Domain skills 编码专业领域知识。Meta skills 用于管理 skill 生态。L2 治理其全生命周期：版本管理、访问范围、依赖解析、晋升，以及使用分析。
- **MCP（Model Context Protocol）：**通用集成协议。外部服务（Gmail、Calendar、Drive）与内部平台服务（L2 skill 仓库、L3 MCP 注册表）都通过 MCP 访问。该协议标准化了工具发现、认证、调用与错误处理，使所有智能体的集成方式一致。

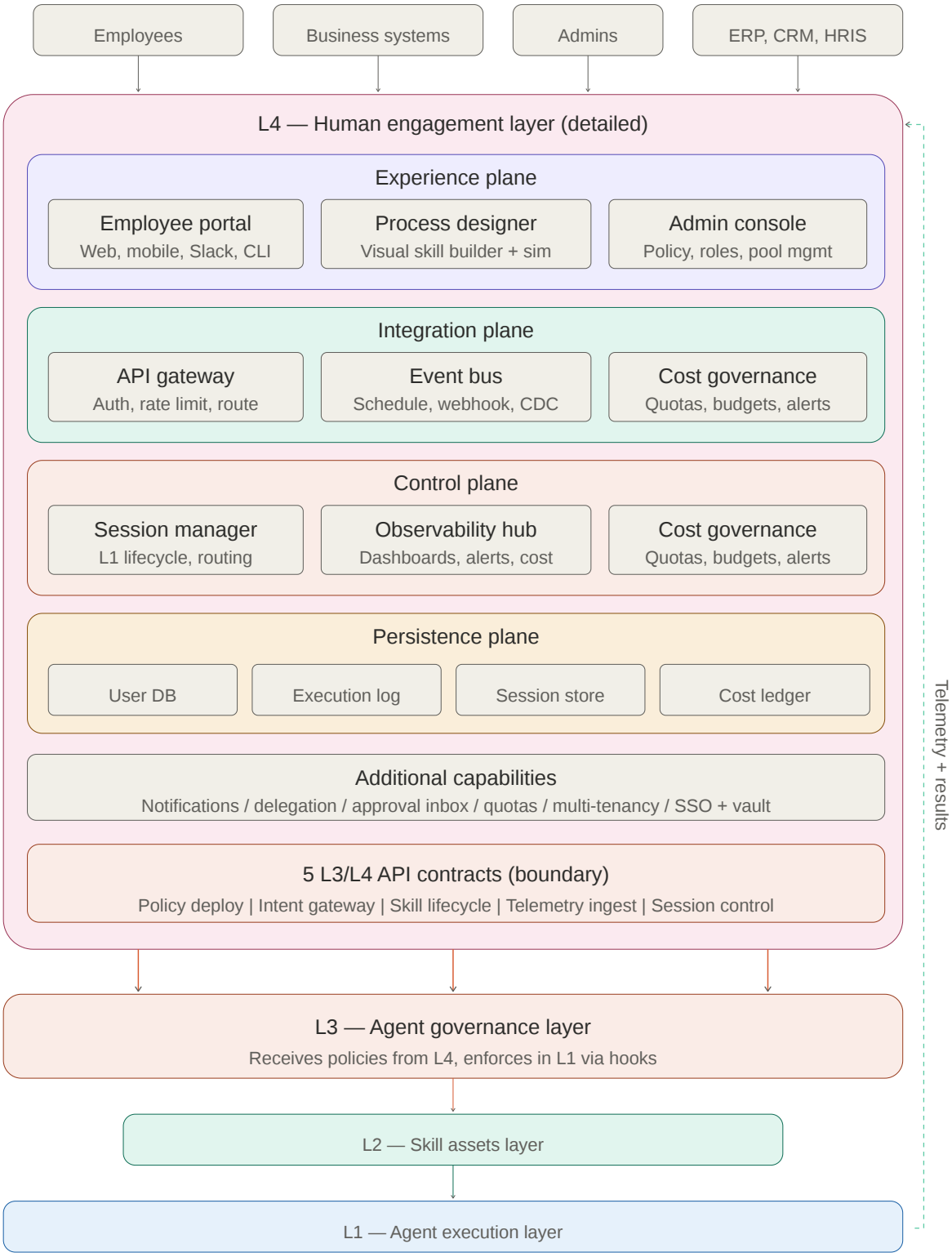
2.3 设计原则

- **治理是统一的：**同一套 L3 受管 hooks 适用于 L1 中的每一种智能体。管理员只需在 L4 控制台部署一次策略，L3 将其编译为 hooks，L1 的 hook bridge 会在 Octo、Claude Code、Aider、Goose 以及任何 LangGraph 智能体上以一致方式强制执行。

- **智能通过 Skills 流动：**L2 中的 Skills 是运行时无关的知识资产。同一个 workflow-skill 可以在 Octo 上运行（通过 kernel boot skill），也可以在 Claude Code 上运行（通过原生 skill 系统），还可以在 LangGraph 智能体上运行（通过 L1 适配器进行 skill→graph 翻译）。
- **Hooks 提供保障，智能体提供判断：**每次都必须发生的事情（策略强制、审计记录、质量校验）由 hooks 负责。需要适应与推理的事情（计划拆解、错误恢复、创造性问题解决）由智能体的智能来完成。不要用其中一方替代另一方。
- **扩展，而非重建：**每个演进阶段都在不修改或不破坏既有能力的前提下增量添加组件。平台按与四层架构对齐的五个阶段逐步扩展 Octo。
- **拒绝优先：**只要任一作用域层级的 hook 拒绝了某个动作，无论其他 hooks 或智能体推理如何决定，该动作都会被阻止。这种分层的“拒绝优先”模型，是 hooks 能够用于企业治理的根本保障。

3. L4 — 人机协作层（Human engagement layer）

L4 是人类用户与企业系统直接交互的一切入口。它被拆分为四个内部平面，通过五个已定义的 API 契约与 L3 通信，并拥有所有持久化状态。



Purple = experience | Teal = integration | Coral = control | Amber = persistence

3.1 体验平面

三类面向用户的应用服务于不同的企业角色：

员工门户。 多渠道接入：Web 应用、移动端、Slack/Teams 内嵌、CLI。通过企业 SSO 处理用户认证，支持跨渠道的对话连续性与会话生命周期管理。门户是一个路由层，负责对来自任意渠道的用户输入进行归一化、完成用户认证、解析其组织角色，并通过会话管理器将消息分发到一个 L1 会话。对话历史由 L4 的会话存储（而非短暂的 L1）持有，从而实现跨会话、跨设备的连续体验。

流程设计器。 面向业务分析师的可视化、表单化工具，用于创建与编辑 workflow-skills，而无需手写 SKILL.md。设计器提供结构化字段，包括工作流名称、触发条件、步骤描述、质量标准、所需的 MCP 连接器以及 hook 配置。它将用户输入编译为 SKILL.md 格式，并通过 skill 生命周期 API 提交到 skill 资产仓库。包含仿真模式：提供一个带只读 MCP 连接器的沙箱 L1 会话，供分析师在提交前测试工作流。

管理控制台。 L4 最关键的应用，提供：策略编辑器（将业务语言规则翻译为受管 hook 配置）、角色管理器（定义 RBAC 角色并映射到 skill 访问与 MCP 权限）、MCP 连接器管理器（注册、配置与监控连接器）、skill 晋升看板（审核并批准 skill 从 draft → tested → reviewed → production）、运行时池管理器（按组织单元启用或禁用运行时、配置选择策略、查看对比分析），以及用户管理界面（分配角色、查看用户活动、管理配额）。

3.2 集成平面

API 网关。所有程序化访问的单一入口。负责处理 OAuth 2.0/SSO 身份认证、限流、请求校验与路由。所有 L2/L3 的接口契约都经由网关进入。支持内部（员工门户）与外部（业务系统）两类访问，并通过认证方式与授权范围进行区分。

事件总线。接收来自外部系统的业务触发，并将其翻译为结构化意图，以便通过 L3 分发。三类触发：定时任务（用于周期性工作流的 cron 表达式）、Webhook（来自 CRM、HRIS、工单平台的 HTTP 回调）、数据变更事件（数据库触发器、流式处理）。每个事件都会被校验（Webhook 签名校验）、去重（使用事件 ID 或内容哈希防止重复执行工作流），并转换为结构化意图。该意图包含来源身份、事件类型、目标 skill ID、参数，以及用于结果通知的回调 URL。

连接器织网。在企业系统（ERP、CRM、HRIS、Active Directory、数据湖）与 L4 持久化平面之间运行的 ETL/ELT 管道。它不同于 L1 的 MCP 连接器（Octo 在执行期间调用）。连接器织网将组织结构、员工数据与业务指标同步到 L4 的数据库中。

3.3 控制平面

会话管理器。管理 L1 运行时的生命周期。当员工门户或事件总线请求一个会话时，会话管理器将：（1）把请求转发给 L1 的运行时选择器以选择运行时；（2）通过 L3 传递会话初始化载荷（用户上下文、角色、组织单元、受管 hooks、配额），以完成鉴权与 hook 绑定；（3）创建

L1.R 运行时实例；（4）维护用于消息路由的会话句柄；（5）在重连时，从 L4 的会话存储中恢复并支持会话续接；（6）在完成或超时后销毁会话，并将状态刷新到会话存储中。对于自动化（无人工交互）的工作流，会话管理器无需用户门户即可创建会话，使 workflow-skills 自主执行。

可观测性枢纽。消费来自 L3 审计服务的遥测数据，并提供仪表盘，用于展示：工作流执行性能（时延、成功/失败率、步骤级追踪）、skill 质量分析（哪些 skill 失败率最高、哪些需要改进）、运行时对比（来自盲测投票的每运行时质量评分、成本效率、时延分布、任务类型亲和性热力图）、MCP 连接器健康度趋势、成本追踪（按用户、团队、工作流统计 token 用量与计算时长）、以及预算告警（当支出模式提示预算超支时触发阈值告警）。该枢纽会把洞察反馈给管理控制台与流程设计器，形成反馈闭环：执行数据（L1）→ 审计遥测（L3）→ 分析（L4）→ skill 改进（回到仓库）。

策略编译器。将管理控制台中的人类可读策略翻译为可执行的受管 hooks。生成 hook 配置 JSON 与强制执行脚本（用于 command handler 的 shell 脚本、用于 HTTP handler 的端点 URL、用于 prompt handler 的提示词、用于 agent handler 的指令）。在部署前于沙箱中用样例事件测试编译产物。通过策略部署 API 以原子方式在所有 L1 实例上发布。支持策略版本管理与回滚。

3.4 持久化平面

L4 拥有四个数据存储，它们共同承载平台的全部持久化状态：

- **用户数据库：**员工画像、组织角色分配、偏好设置与配额使用情况。通过连接器织网与企业身份提供方（Active Directory、Okta、Azure AD）同步。
- **执行日志：**从 L3 审计服务遥测聚合而来的结构化工作流执行历史。提供可查询记录：工作流 X 在时间 Z 为用户 Y 运行，耗时 N 秒完成，产生输出 W，并在步骤 S 发生升级处理。为可观测性枢纽的仪表盘提供数据。
- **会话存储：**跨 L1 会话边界仍可保留的对话历史与会话状态。当一个 L1 会话结束时，会话管理器会把内核状态与对话摘要序列化到此处；当员工再次返回时，会话管理器会用恢复的上下文初始化一个新的 L1 会话。
- **成本台账：**按用户、团队与工作流记录资源消耗：API tokens、计算时长、MCP 调用与存储。为成本仪表盘提供数据，并支持将平台使用成本回收分摊到各部门的计费模型。

3.5 多租户与组织拓扑

L4 对企业的组织层级进行建模：企业 → 事业部（BU）→ 部门 → 团队。策略可以在任一层级定义；较低层级的策略可以收紧，但不能放松较高层级的策略。当会话管理器创建一个 L1 实例时，它会解析员工所属的组织单元，并加载正确的分层受管 hooks 集（企业 → BU → 部门 →

团队）。仓库中的 Skill 也带有组织范围：有些是全局的，有些是 BU 范围的，有些是部门范围的。

3.6 安全架构

L4 负责：通过企业 SSO + MFA 为人类用户进行身份认证，并为系统触发器使用 OAuth 客户端凭据；通过密钥库（HashiCorp Vault、AWS Secrets Manager 或 Azure Key Vault）进行机密管理，用于存放 MCP 连接器凭据、API token 与部署凭据；通过在事件总线的所有外部输入进入 L3/L1 之前进行清洗来防御提示注入；以及通过在 L4 自身的审计轨迹中记录每一次管理员操作（策略部署、Skill 晋升、角色变更）来保证审计链完整性，并与 L3 的执行审计相区分。

3.7 L4 的其他能力

主动通知。由事件总线（员工应知的业务事件）或 L1 执行结果（需要员工采取行动的结果）生成。通知系统处理渠道偏好（推送、邮件、Slack）、优先级（紧急与信息类），以及静默时段。

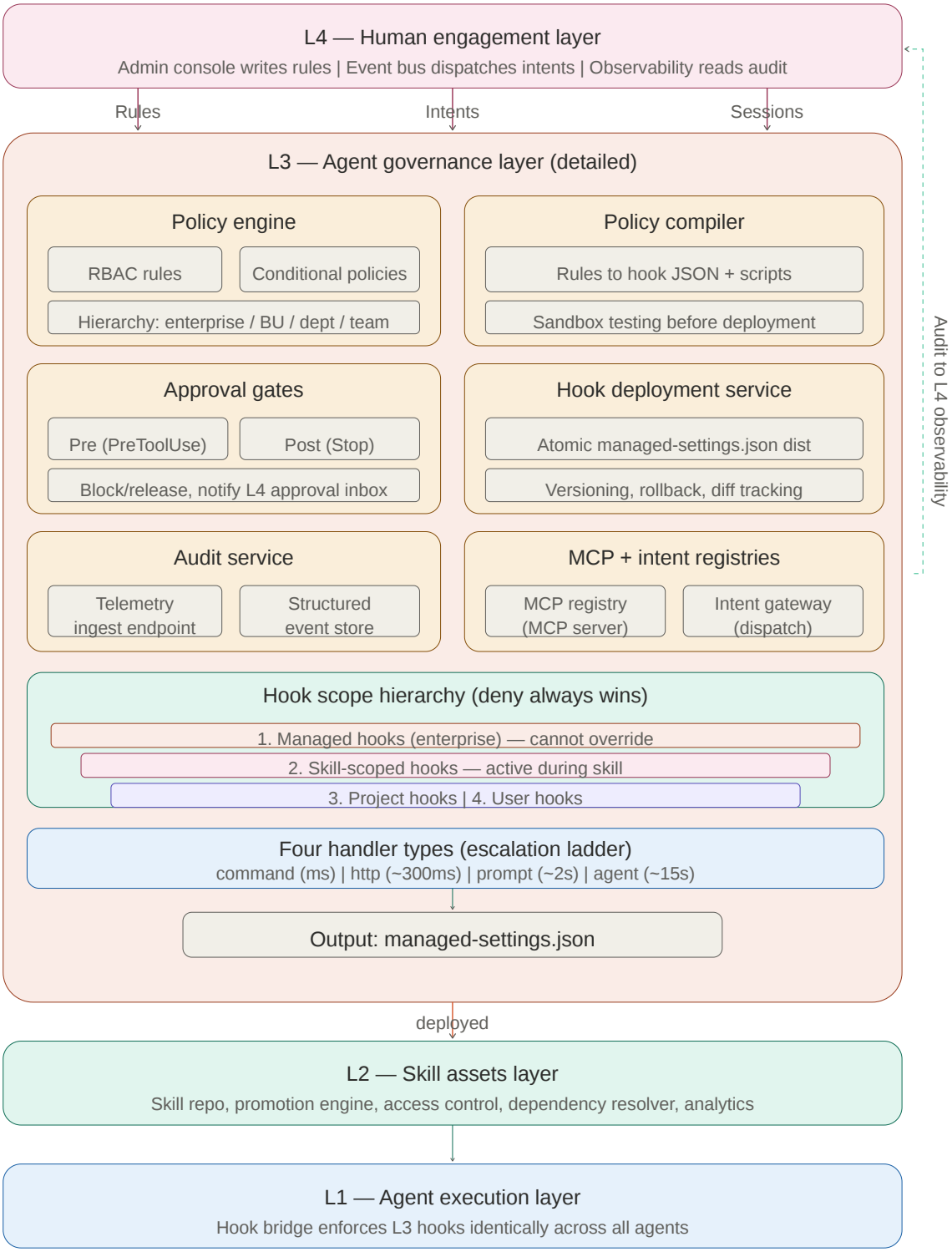
委派与交接。员工可将进行中的工作流转交给同事。会话存储将当前状态序列化为可共享格式；会话管理器为被委派人创建一个预加载该状态的新 L1 会话。

审批收件箱。当某个 hook 或审批闸门暂停执行时，L4 会在员工门户中呈现待处理请求，或作为通知推送，提供上下文并支持一键批准或拒绝。批准操作会调用意图网关 API 以释放被暂停的工作流。

成本治理。配额管理（按用户、团队、部门的限制由 SessionStart hooks 检查配额状态并强制执行）、成本归因（遥测按组织单元聚合）、以及预算告警（来自可观测性枢纽的基于阈值的通知）。

4. L3 — 智能体治理层（Agent Governance Layer）

L3 是一个精简的治理层。它不负责编排工作流，也不做执行决策。它的唯一目的，是强制执行组织策略、治理资产生命周期，并采集审计遥测。其主要的强制机制是受管 hooks。



L3 now has 7 services: policy engine, policy compiler, approval gates, hook deployment, audit service, MCP registry, intent gateway

4.1 策略引擎

将组织策略翻译为受管 hooks。管理员在 L4 管理控制台中声明的策略，由 L3 的策略编译器编译，并通过策略部署 API 下发。策略引擎为所有组织单元维护主配置存储，并支持分层策略作用域：企业级策略适用于所有人；事业部、部门与团队级策略只增加限制。较低层级策略可以收紧，但不能放松较高层级策略。

4.2 审批闸门

以 hook handler 的形式实现，用于检查审批状态。三类闸门包括：执行前闸门（PreToolUse hooks，在工具调用执行前阻止需要审批的调用）、执行中闸门（PostToolUse hooks，在检测到异常或合规问题时阻止继续推进）、执行后闸门（Stop hooks，在确认完成审阅之前阻止结束）。PermissionRequest hook 事件专门支持审批工作流：受管 hooks 可以自动批准安全操作，并将敏感操作升级处理。

4.3 审计服务

通过异步 PostToolUse HTTP hooks 接收来自每个 L1 运行时实例的遥测。存储结构化执行记录：由谁发起、请求了什么、调用了哪些工具、hooks 做了哪些决策、最终结果，以及资源消耗。通过遥测采集 API 将数据馈送给 L4 的可观测性枢纽。审计服务是跨所有运行时的执行历史单一事实来源。

4.4 MCP 注册表

以 MCP 服务器实现，负责管理平台中所有 MCP 连接器的生命周期。对外暴露工具：connector_list、connector_register、connector_health、connector_config、connector_grant、connector_revoke。执行内核在 pre-flight 阶段查询注册表，以验证连接器可用性。SessionStart hook 会将当前 MCP 健康状态注入会话上下文。

4.5 Hook 作用域层级

Hooks 在四个作用域层级部署。任一层级的拒绝（Deny）都会阻断动作，不受其他 hooks 的影响：

- **受管 hooks（企业级）：**由 L3 的策略编译器通过 managed-settings.json 部署。不能被用户或项目覆盖。当启用 allowManagedHooksOnly 时，仅允许受管 hooks。这一层承载 L3 治理策略。
- **Skill 作用域 hooks（frontmatter）：**定义在 skill 的 YAML frontmatter 中。仅在 skill 执行期间生效，skill 结束后自动清理。这是 workflow-skills 强制执行领域规则的方式。
- **项目 hooks：**定义在项目设置中，团队共享，用于强制项目级规范（格式、测试、评审要求）。

- **用户 hooks：**定义在用户设置中，用于个人偏好与便捷自动化。

4.6 四类 hook handler 类型

类型	机制	适用场景	成本 / 时延
command	Shell 脚本：从 stdin 接收 JSON，返回退出码 + stdout	文件路径检查、拒绝清单匹配、格式校验、确定性规则	近乎为零：毫秒级，无需 API 调用
http	POST 到 HTTP 端点，接收 JSON 响应	面向 L3 服务端的集中策略检查、远程校验服务	低：网络往返，~100–500ms
prompt	向轻量级 Claude 模型发送提示词，进行单轮评估	语义质量评估、合规检查、异常检测	中：LLM 推理，~1–3 秒
agent	拉起带工具权限（Read、Grep、Glob）的子智能体以做深度验证	复杂校验：数值对账、全面代码测试	高：多步推理，~5–30 秒

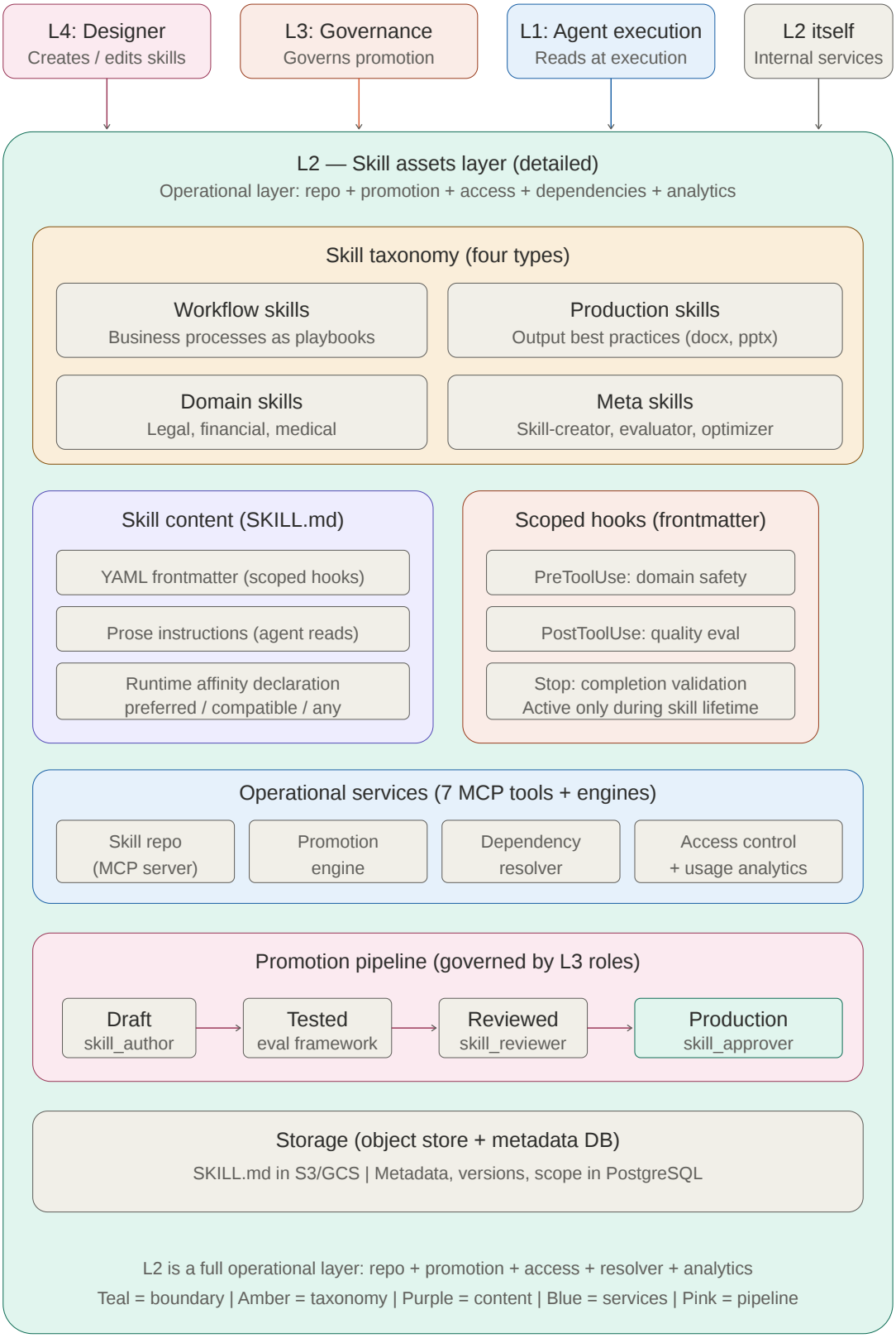
4.7 Hook 生命周期事件

事件	触发时机	架构角色
SessionStart	会话开始（启动、续接、清除、压缩）	上下文注入：用户角色、MCP 健康状态、配额状态。配额强制。
UserPromptSubmit	用户提交提示词，进入处理前	模式路由：SIMPLE 与 KERNEL 分类。策略上下文注入。
PreToolUse	任一工具执行前	主要强制：策略闸门、访问控制、安全。允许/拒绝/询问 + 输入修改。
PostToolUse	工具成功执行后	质量评估（prompt hooks）、异步审计记录（HTTP hooks）、输出校验。
PostToolUseFailure	工具执行失败后	错误遥测、为自适应恢复注入诊断信息、重试逻辑。
PermissionRequest	智能体请求某个工具的权限时	自动批准安全操作（受管 hooks），将敏感操作升级给用户或 L4 审批收件箱。
Stop	智能体完成响应时	最终质量闸门（exit 2 强制智能体继续）。向 L3 审计服务推送遥测。
SubagentStop	子智能体结束时	在纳入主执行流之前校验子智能体输出。

PreCompact	上下文压缩前	在会话历史被压缩前，将执行状态摘要保存到 L4 会话存储。

5. L2 — 技能资产层（Skill assets layer）

L2 负责 Skill 资产的全生命周期：存储、发现、校验、晋升、访问控制、依赖解析、版本管理，以及使用分析。所有与 Skill 内容的交互都必须经过 L2 的运行时服务，并通过 MCP 协议分别供 L4（流程设计器创建 Skills）、L3（治理控制晋升与访问）和 L1（智能体在执行期间读取 Skills）访问。


L4: Designer
Creates / edits skills
L3: Governance
Governs promotion
L1: Agent execution
Reads at execution
L2 itself
Internal services
L2 — Skill assets layer (detailed)
Operational layer: repo + promotion + access + dependencies + analytics
Skill taxonomy (four types)
Workflow skills
Business processes as playbooks
Production skills
Output best practices (docx, pptx)
Domain skills
Legal, financial, medical
Meta skills
Skill-creator, evaluator, optimizer
Skill content (SKILL.md)
YAML frontmatter (scoped hooks)
Prose instructions (agent reads)
Runtime affinity declaration
preferred / compatible / any
Scoped hooks (frontmatter)
PreToolUse: domain safety
PostToolUse: quality eval
Stop: completion validation
Active only during skill lifetime
Operational services (7 MCP tools + engines)
Skill repo
(MCP server)
Promotion
engine
Dependency
resolver
Access control
+ usage analytics
Promotion pipeline (governed by L3 roles)
Draft
skill_author
Tested
eval framework
Reviewed
skill_reviewer
Production
skill_approver
Storage (object store + metadata DB)
SKILL.md in S3/GCS | Metadata, versions, scope in PostgreSQL
L2 is a full operational layer: repo + promotion + access + resolver + analytics
Teal = boundary | Amber = taxonomy | Purple = content | Blue = services | Pink = pipeline

5.1 层级架构

L2 包含六个运营服务，每个服务都可以独立部署与横向扩展：

- **Skill 仓库（MCP server）：**核心的存储与检索引擎。对外暴露七个 MCP 工具：skill_search（按名称、标签、描述、亲和性发现）、skill_read（读取内容，包含 frontmatter hooks）、skill_list_versions（版本历史，包含差异与作者）、skill_submit_draft（提交新版本草稿；校验作者角色）、skill_promote（推进生命周期状态；需要相应角色权限）、skill_dependencies（追踪 Skill 之间的依赖关系）、skill_usage（按运行时统计调用频次、成功率与失败模式）。所有 Skill 内容存放在对象存储（S3/GCS），元数据存放在 PostgreSQL。
- **晋升引擎（Promotion engine）：**管理受治理的生命周期流水线：draft → tested → reviewed → production。每一次迁移都需要来自 L3 的基于角色授权（skill_author 提交，评估框架测试，skill_reviewer 评审批准，skill_approver 最终晋升）。引擎强制执行迁移规则：不得从 draft 直接跳到 production；不得在没有显式回滚的情况下进行降级；不得在未通过所有自动化测试时晋升。每一次迁移都会记录到 L3 的审计服务中，包含审批者身份、时间戳，以及相对上一版本的 diff。
- **访问控制服务（Access control service）：**对所有 Skill 操作实施组织范围控制。Skill 可被限定在企业全局（所有人可见）、事业部、部门或团队范围。访问控制与 L3 的 RBAC 协同工作，用于判断用户是否可以读取、编辑、提交、评审或批准 Skill。当 L1 在执行过程中请求读取 Skill 时，访问控制服务会校验该用户的组织单元是否符合 Skill 的 scope，验证通过后才返回内容。
- **依赖解析器（Dependency resolver）：**追踪 Skill 之间的依赖并校验兼容性。当某个 Skill 引用了其他 Skill（例如某个 workflow-skill 调用用于法务审查的 domain-skill）时，解析器会确保所有依赖都处于兼容版本、对请求方可访问，并且处于 production 状态。在晋升阶段，解析器还会检查晋升或废弃某个 Skill 是否会破坏下游依赖者。
- **版本管理器（Version manager）：**处理语义化版本、分支与回滚。每一次 Skill 编辑都会生成一个新版本，并保留完整 diff。版本管理器支持回滚到任意历史版本、版本锁定（workflow-skill 可将依赖固定到某个版本）、以及废弃告警。它维护完整的编辑历史，包含变更人、变更内容与时间。
- **使用分析引擎（Usage analytics engine）：**收集来自 L1 遥测（经由 L3 审计服务）的调用数据，并生成按 Skill、按运行时的指标：调用频次、成功/失败率、平均执行时长、token 消耗、用户满意度分数（来自盲盒投票）、以及失败模式分析。这些指标会进入 L4 的可观测性枢纽，为 Skill 改进决策提供依据。该引擎还会识别长期未使用的 Skill 作为潜在废弃候选，并发现高失败率 Skill 以触发质量评审。

5.2 Skill 分类（Skill taxonomy）

四类 Skill：存储与治理方式一致，但用途不同：

Skill 类型	用途	Hook 集成方式
Workflow skills	将业务流程编码为“智能作战手册”：步骤、质量标准、错误处理、资源声明等，用于替代僵化的 YAML 流水线。	在 frontmatter 中携带作用域 hooks；用 PostToolUse prompt hooks 做质量评估；用 Stop hooks 做完成性校验；用 PreToolUse hooks 做领域安全约束。
Production skills	面向输出类型的最佳实践：docx、pptx、xlsx、pdf 等生成规范。智能体在交付物生产前读取的操作指南与注意事项。	用 PostToolUse command hooks 做格式校验（例如生成 docx 后运行 validate.py）；用 PreToolUse hooks 做模板与输入前置校验。
Domain skills	专业领域知识：法律术语、财务合规、医学编码标准、监管要求等。	用 PreToolUse prompt hooks 做合规检查；用 PostToolUse hooks 做术语与表述一致性校验。
Meta skills	“关于 skill 的 skill”：例如 skill-creator（迭代评估与改写）、description optimizer（提升触发命中率）、dependency analyzer（依赖分析）。	用 agent hooks 做深度评估；用 PostToolUse hooks 跟踪与记录评估结果。

5.3 Skill 内容结构

每个 Skill 都是一个包含三个部分的 SKILL.md 文件：

- **YAML frontmatter：**声明作用域 hooks（仅在该 Skill 执行期间生效的 PreToolUse、PostToolUse、Stop handlers）、运行时亲和性（preferred_runtime、compatible_runtimes，或不声明亲和性）、访问范围（组织单元）、依赖项（所需的其他 Skills）以及元数据（作者、版本、标签、描述）。
- **文字说明（Prose instructions）：**智能体会读取并遵循的自然语言指令。面向“聪明的读者”编写，描述意图、质量标准、边界情况，以及适配策略。文字说明是 Skill 的“智能”部分，它定义什么是好结果，以及如何应对意外情况。
- **运行时亲和性声明：**Skill 可以指定 preferred_runtime（例如依赖 kernel 的工作流更适合 Octo）、compatible_runtimes（支持该 Skill 要求的一组运行时），或不声明亲和性（只要运行时具备所需工具即可运行）。L1 的运行时选择器在选择运行时时会考虑该亲和性；不声明亲和性的 Skill 可移植性最高，应优先选择。

5.4 Skill 中的作用域 hooks

Workflow-skills 解决了智能性与可靠性之间的张力。Skill 的文字说明告诉智能体要做什么以及如何适配；Skill 的 frontmatter hooks 则保证某些检查每次都必然发生。智能与保障在同一

份资产中共存，各自承担最擅长的部分。

在 Skill frontmatter 中最常用的三个 hook 事件是：PreToolUse（领域安全，例如“在工作时间阻止写入生产数据库”）、PostToolUse（质量评估，例如“审查输出中是否存在超过 25% 方差的异常；如发现则阻断”）、Stop（完成性校验，例如“验证最终文档包含所有必需章节”）。Skill 作用域 hooks 在 Skill 被加载时激活，在 Skill 完成时自动失效。它们会与 L3 的受管 hooks 组合：两者并行触发，任一方拒绝（Deny）都会阻断动作。

5.5 运行时亲和性

Skill 可以声明运行时亲和性：preferred_runtime（该 skill 在某个特定运行时上效果最佳，例如依赖内核能力的工作流更适合 Octo）、compatible_runtimes（支持该 skill 要求的运行时列表），或不声明亲和性（任何支持所需工具的运行时均可）。L1.0 的运行时选择器在选择运行时时会考虑 skill 亲和性。未声明亲和性的 skill 可移植性最高，应尽量优先。

5.6 仓库 MCP 工具

通过 MCP 暴露七个工具：

- skill_search（按名称、标签、描述、亲和性发现 skills）
- skill_read（读取内容，包含 frontmatter hooks）
- skill_list_versions（版本历史，包含差异与作者）
- skill_submit_draft（提交草稿新版本；L2 校验作者角色）
- skill_promote（推进生命周期状态迁移；需要相应角色权限）
- skill_dependencies（追踪 skills 之间的依赖关系）
- skill_usage（按运行时统计调用频次、成功率与失败模式）。

5.7 晋升流水线（Promotion pipeline）

Skill 会经过受治理的生命周期，包含四个阶段与三道闸门：

- **Draft：**由具备 skill_author 角色的用户提交。系统对 Skill 做语法级校验（解析 YAML frontmatter、检查 hook 声明、将运行时亲和性与已知运行时清单核对）。仅作者可在沙箱会话中测试。
- **Tested：**晋升引擎通过评估框架测试 Skill：自动化测试用例（如有）、skill-evaluator 元 Skill 分析、以及在沙箱中用样例输入执行。测试结果会附在该版本上。未通过自动化测试的 Skill 不可晋升。
- **Reviewed：**具备 skill_reviewer 角色的用户审阅 Skill 内容、测试结果、hook 声明与依赖图。评审者可批准、带反馈拒绝，或要求修改。评审决策会记录理由。

- **Production：**具备 skill_approver 角色的用户做最终批准。Skill 会对组织角色与其访问范围匹配的所有用户可用。Production 状态的 Skill 不可变更，任何修改都必须通过完整流水线创建新版本。

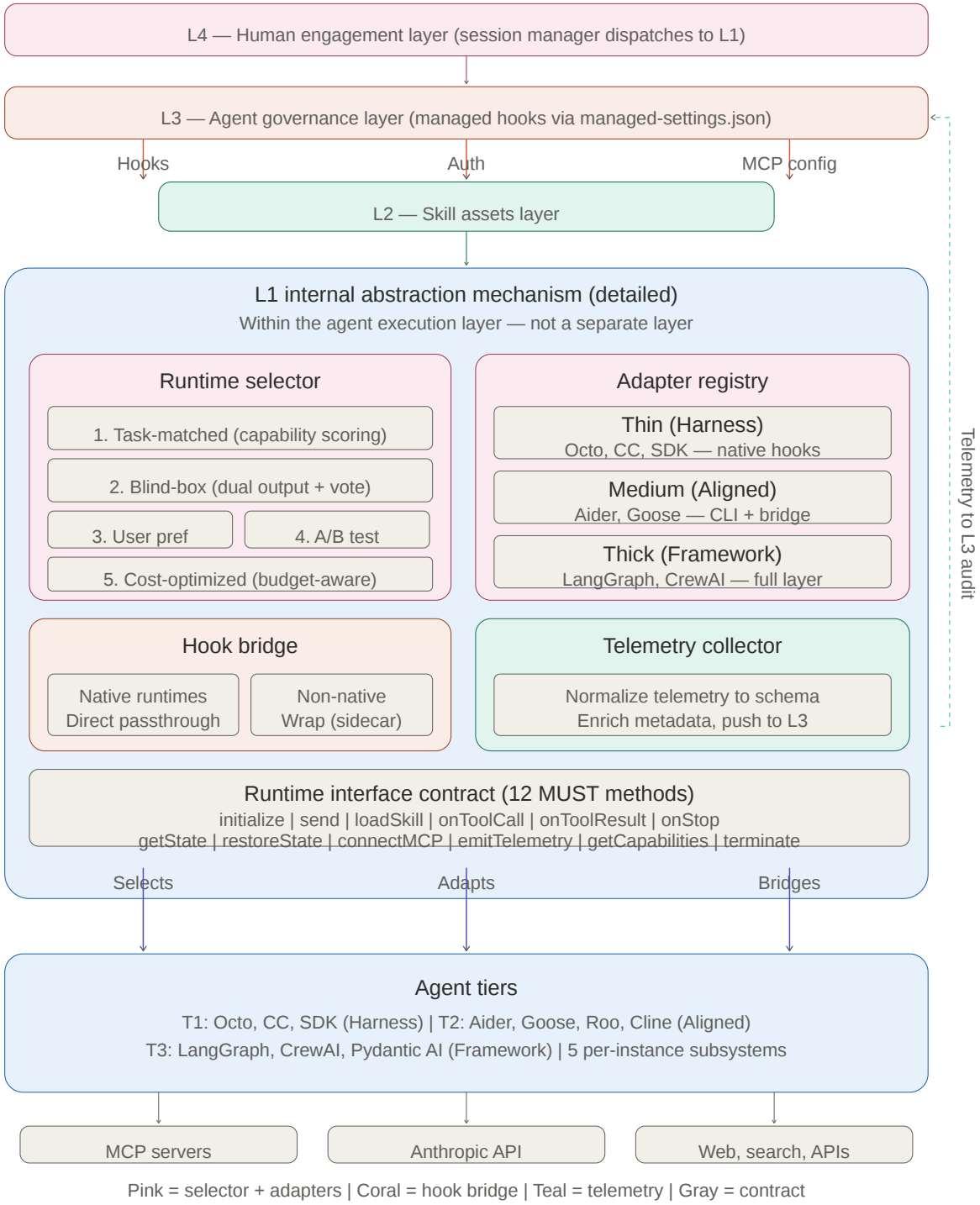
5.8 L2 与相邻层的接口

L2 通过已定义的机制与相邻层通信：

- **L4 → L2：**L4 的流程设计器通过 Skill 生命周期 API（L3/L4 契约 3）创建与编辑 Skills，L3 将其路由到 L2 的晋升引擎与仓库。
- **L3 → L2：**L3 的治理能力控制 L2 的晋升流水线（基于角色的闸门）、访问范围（组织层级），以及审计记录（每一次 Skill 操作都会被记录）。L3 的 managed-settings.json 可按 ID 引用 L2 Skills。
- **L2 → L1：**L1 的智能体通过 Skill 仓库 MCP server 访问 L2。在执行期间，智能体调用 skill_search 发现相关 Skills，调用 skill_read 加载内容，调用 skill_dependencies 解析所需子 Skills。L2 的访问控制会对每一次读取请求按用户的组织范围进行校验。
- **L2 ← L1（经由 L3）：**来自 L1 的执行遥测经由 L3 的审计服务进入 L2 的使用分析引擎，从而闭环：执行数据影响质量指标，质量指标驱动改进优先级。

6. L1 内部抽象机制

L1 的内部抽象机制实现了与智能体运行时（Agent Runtime）无关的运行。L4、L3 和 L2 通过该机制与 L1 交互，而不是与具体智能体直接耦合。它包含四个组成部分。



6.1 运行时选择器 (Runtime Selector)

当 L4 的会话管理器请求创建新会话时，运行时选择器会从运行时池中选择一个具体运行时。可支持五类选择策略：

- **任务匹配（Task-matched）：**分析请求（或基于用户角色/历史推断的工作负载画像），与运行时能力清单（capability manifests）匹配。编码任务路由到 Claude Code；文档工作流路由到 Octo；研究/检索类任务路由到 LangGraph 智能体等。
- **随机盲盒（Random blind-box）：**同时分配两个运行时并产出两份结果，用户在不知道来源的情况下对结果投票（up/down）。用于生成成对对比数据，从而形成经验驱动的质量评分。代价：抽样会话的执行成本翻倍。
- **用户偏好（User preference）：**员工显式选择偏好的运行时。可受策略约束（管理员可对不同角色限制可用运行时）。
- **A/B 测试（A/B testing）：**受控随机分配：一定比例会话路由到实验运行时，其余路由到默认运行时，用于安全灰度发布。
- **成本优化（Cost-optimized）：**在满足任务画像的候选运行时中选择最便宜的一个。成本信号来自能力清单中的 token 成本与计算成本，可与“任务匹配”组合使用。

6.2 适配器注册表 (Adapter registry)

每一种运行时都对应一个适配器（adapter）——它是平台运行时接口契约与该运行时原生 API 之间的“垫片/转译层”。不同运行时层级的适配器厚度不同：

- **Harness 类运行时（薄适配器）：**Octo、Claude Code、Claude Agent SDK。原生支持 hooks、MCP、skills。适配器主要负责会话生命周期与载荷（payload）翻译。
- **对齐型运行时（中等适配器）：**Aider、Goose、Roo Code CLI、Cline CLI。可能部分支持 MCP，但不原生支持 hooks。适配器通过 hook bridge 桥接 hooks，翻译 skill 格式，并对输出做归一化。
- **框架型运行时（厚适配器）：**LangGraph、CrewAI、Pydantic AI、AutoGen。具有自定义执行模型。适配器需要把 skills 翻译为框架的内部形式（如 LangGraph 节点、CrewAI crew 定义），并桥接 MCP、桥接 hooks，同时归一化输出。

6.3 Hook 桥接器 (Hook bridge)

Hook 桥接器的目标是确保 L3 治理对所有运行时“同样生效”。

对于原生支持 hooks 的运行时：受管 hooks 直接加载到运行时的 hook 系统中，进程内执行，几乎零额外开销。

对于不原生支持 hooks 的运行时：桥接器需要在外部包装每一次工具调用。调用序列为：框架决定调用某工具 → bridge 拦截 → 运行 PreToolUse hooks → 若允许则转发至工具 → 工具执行 → bridge 拦截结果 → 运行 PostToolUse hooks → 若未被阻断则把结果返回给框架。

为降低外部拦截带来的延迟开销，bridge 应与运行时同部署（如同一容器网络）。

6.4 遥测采集器 (Telemetry collector)

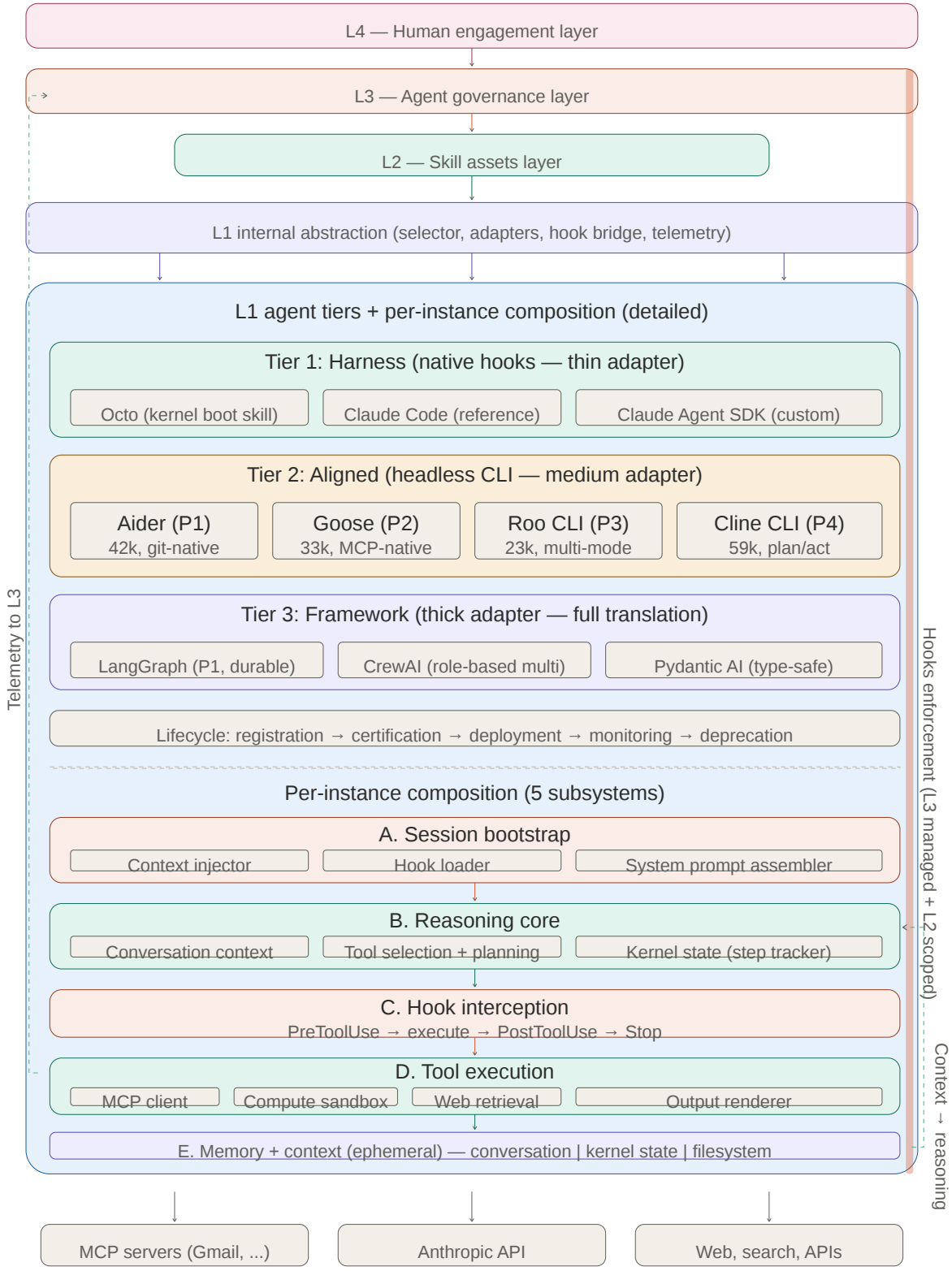
遥测采集器负责把来自所有运行时的遥测统一归一到平台标准 schema，并推送到 L3 审计服务。对于 hook-native 运行时：遥测通过异步 PostToolUse hooks 流出。对于 bridge 运行时：hook bridge 在拦截工具调用时捕获遥测。采集器在转发前会用 runtime ID、session ID、用户身份等元信息对事件进行补全与富化。

6.5 运行时接口契约

任何运行时要加入运行时池，都必须（通过其适配器）实现以下方法：

Method	Level	Description
initialize(payload)	必须（MUST）	接收会话初始化载荷（用户上下文、角色、受管 hooks、配额等），返回会话句柄。适配器将载荷翻译为运行时的原生配置。
send(message)	必须（MUST）	接收用户消息或结构化意图，返回流式响应。适配器将平台消息格式映射为运行时输入格式。
loadSkill(content)	必须（MUST）	加载 workflow-skill 的 SKILL.md 内容，并激活作用域 hooks。对于框架型智能体：适配器将其转换为图/crew 等框架配置。
onToolCall / onToolResult	必须（MUST）	发出 PreToolUse/PostToolUse 事件。原生支持：透传。桥接方式：由 hook bridge 包装工具调用。
onStop()	必须（MUST）	运行时结束时发出 Stop 事件，并支持 exit-2 阻断机制（强制智能体继续工作）。
getState / restoreState	必须（MUST）/ 建议（SHOULD）	序列化/恢复会话状态，用于持久化到 L3 会话存储，从而支持跨会话连续性。
connectMCP(servers)	必须（MUST）	连接 MCP 服务器（skill 仓库、MCP 注册表、外部服务等）。对不支持 MCP 的运行时：由适配器做协议翻译。
emitTelemetry()	必须（MUST）	以平台标准 schema 发出标准化遥测事件；采集器归一并转发到 L3。
getCapabilities()	必须（MUST）	返回能力清单：模型、上下文窗口、工具、原生 hooks/MCP/skills 支持情况、成本、优势与劣势。
terminate()	必须（MUST）	清理资源，发出 SessionEnd，并在退出前冲刷（flush）所有异步遥测。

7. L1 — 智能体执行层 (Agent excution layer)



Teal = Harness / tools | Amber = Aligned | Purple = Framework / abstraction | Coral = hooks

7.1 运行时分层（Runtime tiers）

Tier 1：Harness 智能体（原生兼容）

构建在 Anthropic 的基础设施之上。原生 hooks（21 个事件，4 类 handler）、原生 MCP、原生 skills、managed-settings 分层体系。适配器很薄。几乎没有治理开销。

- **Claude Code：**参考级 harness。基于终端的编码智能体。完整的 managed-settings，支持 allowManagedHooksOnly。支持子智能体、带 frontmatter hooks 的 skills。编码能力最强。由 Claude Opus 4.6 驱动。
- **Octo：**在 Claude Code 基础上扩展了 kernel boot skill。面向企业个人助理场景。包含模式路由与结构化执行协议（意图解析器、资源协调器、执行循环）。主要的通用运行时。
- **Claude Agent SDK：**Anthropic 用于构建自定义 harness 智能体的 SDK。企业可构建领域智能体（财务智能体、法务智能体），并继承原生 hooks/MCP。支持进程内 MCP server 模型与生命周期 hooks。

Tier 2：对齐型智能体（无界面，轻—中等适配器）

原生无界面（headless）的 CLI 智能体，可作为独立的服务端进程运行。平台兼容性部分满足。重要说明：该层中的部分智能体（Cline、Roo Code）主要形态是 VS Code 扩展，只有它们的 CLI 变体才符合服务端池的要求。VS Code 扩展可作为客户端运行时与之共存，并由本地设置文件中的受管 hooks 进行治理。

- **Aider（42k stars）：**原生 headless CLI。Git 原生（每次变更自动提交）。支持 100+ 模型。在基准中速度最快、token 效率最高。执行循环简单，便于实现 hook bridge。Tier 2 池扩展优先级 1。
- **Goose（33k stars）：**原生 headless。Block/Square、Linux Foundation 治理。原生支持 MCP，本地优先，可扩展。透明的执行模型适合 hook bridge。优先级 2。
- **Roo Code CLI（23k stars）：**VS Code 扩展的 CLI 变体。多模式（architect/code/debug/orchestrator）可映射到 kernel 执行协议。支持 skills 与 checkpoints。原生支持 MCP。优先级 3。
- **Cline CLI（59k stars）：**VS Code 扩展的 CLI 变体。Plan/Act 模式，其“先审批再执行”的理念与 hook 模型一致。原生支持 MCP。Cline Teams 增加了 RBAC。优先级 4。
- **OpenCode：**原生 headless。跨平台、模型无关。架构简洁。优先级 5。

Tier 3：框架型智能体（需要较重适配）

属于构建智能体的 AI 框架（库），而非开箱即用的智能体。适配它们需要先用该框架构建一个智能体，然后再用平台适配器进行封装。

- **LangGraph（GA v1.0）：**生产成熟度最佳。图式编排。支持带 checkpoint 的持久执行（崩溃恢复）。存在模型前后置 hooks（与我们的 tool hooks 接近）。LangSmith 提供可观测性。是长时运行、可崩溃恢复工作流的最强候选。Tier 3 优先级 1。
- **CrewAI（46k stars）：**基于角色的多智能体。原生 MCP + A2A。社区最大。日执行量 1200 万以上。适合“团队结构化”的任务。其角色模型与 kernel 模式差异较大，需要较重的 skill 翻译。
- **Pydantic AI（15k stars）：**类型安全、Python 代码风格干净。原生 MCP。智能体模型简单。需要完整的 hook bridge + skill 翻译。
- **Microsoft Agent Framework：**AutoGen + Semantic Kernel 合并而来。RC 时间为 2026 年 2 月。图式工作流、A2A、MCP。Azure 原生企业特性。与 Azure 强耦合。
- **Google ADK：**多模态、A2A、MCP。与 Vertex AI 集成。与 GCP 耦合。需要完整的 hook bridge。

Tier 4：社区智能体（暂缓）

OpenClaw（257k stars）是生活助理，拥有自有 skill 系统，不支持 hooks，也不支持 MCP，并存在严重安全漏洞（CVE-2026-25253）。AutoGen 处于维护模式，几乎没有安全机制。两者均被暂缓纳入运行时池。

7.2 运行时池中的生命周期

1. 注册（提交适配器 + 能力清单）
2. 认证（针对接口契约的自动化测试 + 与既有运行时进行盲盒质量对比测试）
3. 部署（为特定组织单元启用，并配置盲盒抽样比例）
4. 监控（通过遥测、盲盒投票、错误率进行持续质量评分）
5. 退役（当被替代或质量跌破阈值时，从选择器候选中移除）。

7.3 单实例内部组成

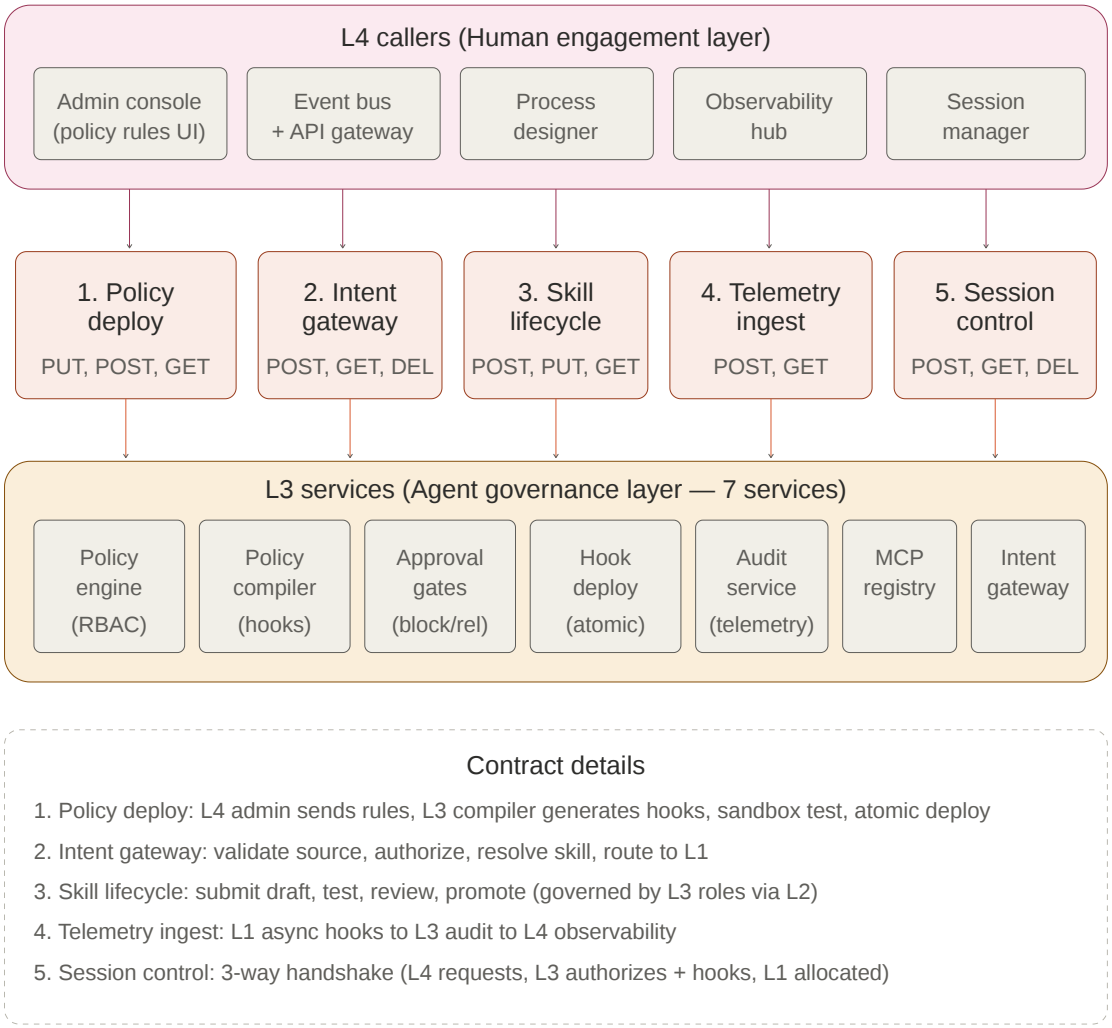
每个运行时实例，无论实现方式如何，都包含五个内部子系统：

- **会话引导平面：**上下文注入器（格式化会话初始化载荷）、hook 加载器（激活受管 hooks 或向桥接器注册）、系统提示词组装器（将 kernel boot skill、组织上下文、偏好与续接状态组装为完整提示词）。为便于调试，组装过程必须是确定性的。

- **推理核心：**决定行动的 LLM 或框架引擎。维护对话上下文（消息历史 + 工具结果）与内核状态（结构化步骤跟踪器）。上下文窗口管理：PreCompact hook 在压缩前保存状态。对于 harness 智能体：使用带 kernel boot skill 的 Claude。对于框架：使用其原生执行循环。
- **Hook 拦截层：**位于推理与工具执行之间。每次工具调用都触发 PreToolUse → 执行 → PostToolUse。受管 hooks、skill 作用域 hooks、项目 hooks、用户 hooks 都会触发；任一拒绝（Deny）都会阻断。对于非原生运行时：hook bridge 在外部执行拦截。
- **工具执行平面：**四类工具族：MCP 客户端（统一访问所有 MCP 服务器）、计算沙箱（容器化 Linux，临时文件系统）、Web 获取（搜索、抓取、图片搜索）、输出渲染器（制品、文件、API-in-API）。MCP 客户端维护连接池与按服务器的健康追踪。
- **记忆与上下文平面：**三种状态形态：对话记忆（完整历史，可能被压缩）、内核状态（结构化步骤跟踪器，跨压缩保留）、工作文件系统（/home/claude，临时）。反馈回路：推理 → 行动 → 观察 → 推理。L1 数据只会在写入 L2（hooks）或 L3（MCP）后跨会话保留。

8. L3/L4 接口：五个 API 契约

五个 REST API 契约定义了 L3 与 L4 之间的全部通信。禁止任何其他跨边界通信。每个契约都必须经过 L3 的 API 网关，以完成身份认证、限流和版本管理。



Contract details

1. Policy deploy: L4 admin sends rules, L3 compiler generates hooks, sandbox test, atomic deploy
2. Intent gateway: validate source, authorize, resolve skill, route to L1
3. Skill lifecycle: submit draft, test, review, promote (governed by L3 roles via L2)
4. Telemetry ingest: L1 async hooks to L3 audit to L4 observability
5. Session control: 3-way handshake (L4 requests, L3 authorizes + hooks, L1 allocated)

8.1 契约 1：策略部署

调用方：L4 管理控制台 + L3 策略编译器。

端点：

- PUT /v1/policies/managed-hooks（将编译后的 hooks 以原子方式部署到所有 L1 实例）
- POST /v1/policies/test（在沙箱中用样例事件测试 hooks）
- GET /v1/policies/versions（列出所有策略版本及其差异与作者）
- POST /v1/policies/rollback（回滚到某个历史 managed-settings 版本）。

实现方式：策略编译器生成 hook JSON 与强制执行脚本，在一个沙箱化的 L1 会话中测试产物，并通过 managed-settings 分发机制部署。部署必须是原子的：要么所有实例都收到更新，要么整体回滚。

8.2 契约 2：意图网关

调用方：事件总线 + API 网关。

端点：

- POST /v1/intents/dispatch（将业务事件翻译为意图，校验来源身份，按策略授权，解析目标 skill，并路由到会话管理器以分配 L1 资源）
- GET /v1/intents/{id}/status（异步轮询意图执行进度）
- POST /v1/intents/{id}/approve（在人类批准后释放被暂停的工作流）
- DELETE /v1/intents/{id}/cancel（中止处于待处理或运行中的意图）。

网关对每一次 dispatch 执行三项检查：来源认证、策略授权、skill 解析。

8.3 契约 3：技能生命周期

调用方：流程设计器 + 管理控制台。

端点：

- POST /v1/skills/submit（在仓库中创建 draft；L2 校验提交者具有 skill_author 角色）
- PUT /v1/skills/{id}/promote（推进生命周期：draft → tested → reviewed → production；每次状态迁移都需要相应角色权限）
- GET /v1/skills/{id}/versions（版本历史，包含差异、作者与变更原因）
- GET /v1/skills/search（按名称、标签、描述、运行时亲和性发现 skills）。

Skill 的晋升流水线由 L3 治理：谁能提交、谁能评审、谁能批准进入 production。

8.4 契约 4：遥测采集（Telemetry ingest）

调用方：L3 审计服务（接收来自 L1 hooks）。

端点：

- POST /v1/telemetry/events（以标准化 schema 接收异步 hook 遥测事件）
- GET /v1/telemetry/query（为可观测性仪表盘提供结构化查询：按用户、skill、时间范围、结果、运行时）、
- GET /v1/telemetry/aggregate（用于成本追踪、skill 使用分析、运行时对比的汇总）。

遥测 schema 捕获：session ID、runtime ID、用户身份、每一次工具调用（工具、输入、输出、耗时）、每一次 hook 事件（触发了哪些 hooks、决策是什么）、最终结果、资源消耗，以及盲盒投票。

8.5 契约 5：会话控制（Session control）

调用方：员工门户 + 会话管理器。

端点：

- POST /v1/sessions/create（三方握手：L3 请求会话 → L2 校验用户角色并附加受管 hooks → L1.0 选择运行时并分配实例）
- GET /v1/sessions/{id}（查询会话状态与元数据）
- POST /v1/sessions/{id}/message（将用户输入发送到 L1，并将响应流式返回）
- DELETE /v1/sessions/{id}（销毁会话并将状态刷新到会话存储）。

该契约是三层的汇合点：L4 发起，L3 治理，L1 执行。会话创建后，消息在 L3 与 L1 之间直接流式传输，同时 L3 的 hooks 在运行时内联拦截并强制执行每一次工具调用。

8.6 会话创建：三方握手（Three-way handshake）

会话创建序列是最关键的集成点。

（1）员工打开门户或事件总线收到事件。

（2）L3 对用户或来源进行认证，并解析其组织角色与组织单元。

（3）L3 调用 POST /v1/sessions/create，携带用户上下文、任务画像与请求的运行时偏好。

（4）L3 按策略校验用户角色，解析该组织单元分层的受管 hooks 集，并将其绑定到会话。

（5）L1.0 的运行时选择器基于任务画像、skill 亲和性与选择策略选择运行时。

（6）L1 分配运行时实例，会话引导平面启动（上下文注入器、hook 加载器、系统提示词组装器），并触发 SessionStart hook 注入上下文。

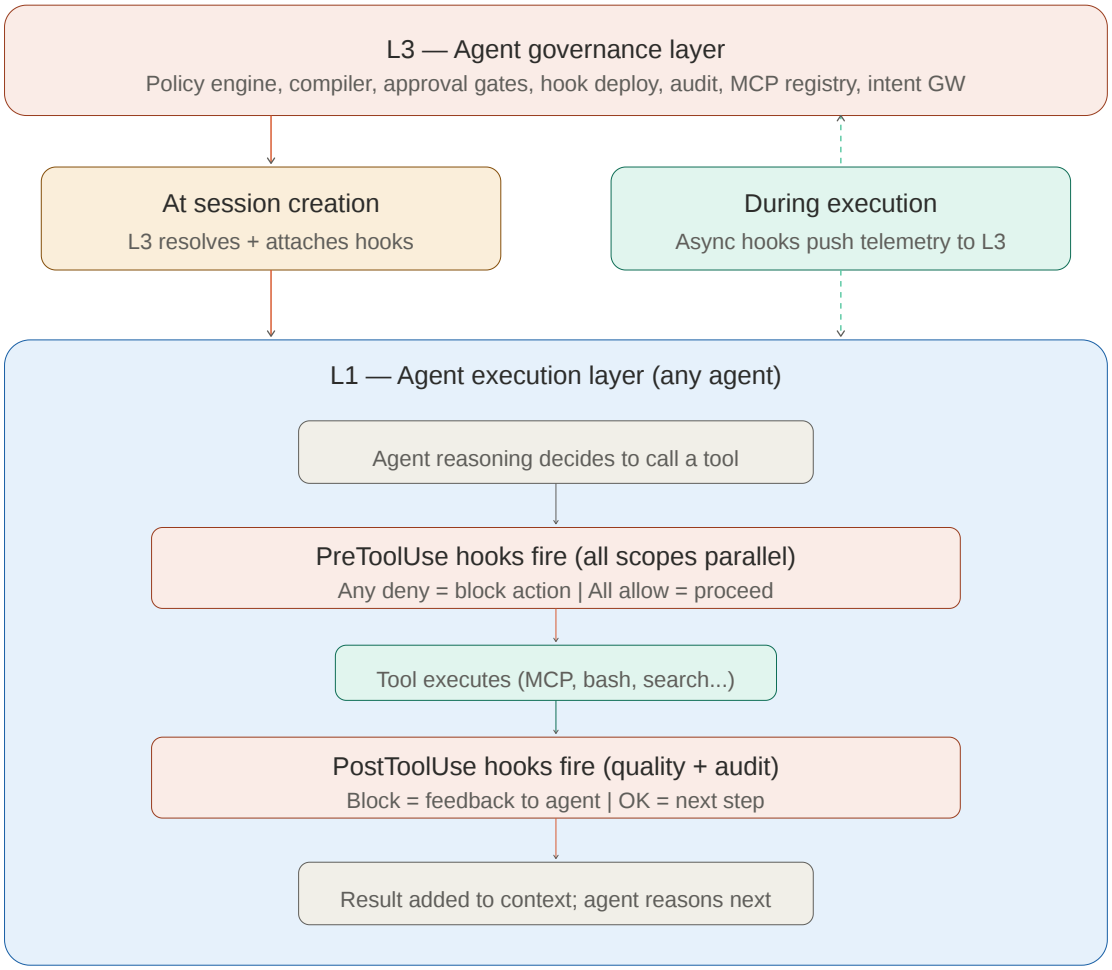
（7）L3 将会话句柄返回给 L4。

（8）L4 将门户连接到会话并开始流式传输消息。

创建完成后，消息流为 L3 ↔ L1，同时 hooks 在内联路径上执行 L3 治理与强制。

9. L1/L3 接口：hooks 作为连接机制

L1/L3 接口在本质上不同于 L3/L4 接口。L3/L4 通过显式的 REST API 契约进行通信。L1/L3 则通过 hooks 进行通信：hooks 是嵌入式的强制机制，在会话创建时被注入到 L1 中，并在执行过程中以内联方式运行。



No L3 API calls during execution. Hooks are embedded at creation, fire inline.
Native agents: zero latency. Bridged agents: hook bridge wraps externally.

9.1 L3 治理如何触达 L1

在会话创建时（契约 5 的三方握手），L3 会为用户所属组织单元解析分层的受管 hooks，并将其附加到会话上。对于 harness 智能体，这些 hooks 会加载到运行时的原生 hook 系统中。对于其他运行时，它们会注册到 L1 的 hook bridge。自此之后，L3 治理被嵌入到 L1 中——每条消息或每次工具调用都不需要额外的 L3 API 调用。Hooks 在运行时的执行循环中以内联方式触发，使得对原生运行时的强制开销近乎为零时延。

9.2 L1 如何回报给 L3

L1 通过异步的 PostToolUse HTTP hooks 将遥测推回 L3。每次工具调用之后，这些 hooks 都会触发，将执行上下文以 POST 的方式发送到 L3 的审计服务端点，然后立即返回而不阻塞执行。Stop hook 会触发一次最终的遥测推送，包含完整的执行摘要。对于采用 hook bridge 的运行时，由 bridge 负责遥测采集，并通过同一个 L3 审计端点进行推送。

9.3 skills 如何桥接 L1 与 L3

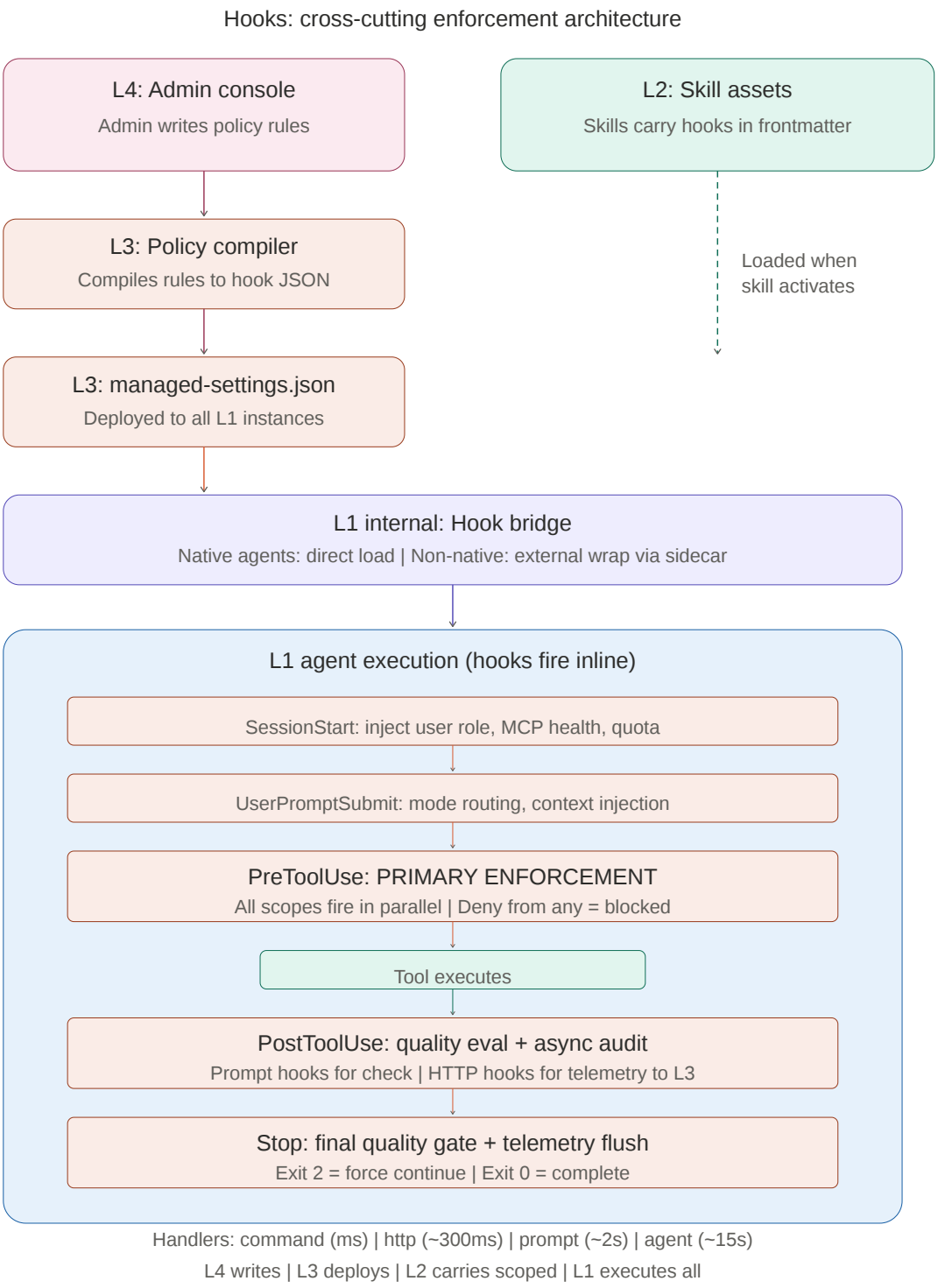
当执行过程中加载某个 workflow-skill 时，该 skill 的作用域 hooks 会与受管 hooks 一起在 L1 中被激活。这些 skill 作用域 hooks 受 L3 治理（skill 的晋升状态、访问范围以及 hook 配置都由 L3 的治理机制管理），但它们在 L1 中执行，并与运行时的工具调用路径内联运行。Skill 仓库就是桥梁：L3 治理仓库中的内容，L1 从仓库读取并使用。

9.4 审批闸门如何跨越 L1/L3 边界

当某个 hook 检测到需要人工审批的条件时（例如某个 PostToolUse prompt hook 发现超过阈值的财务异常），它会阻断执行并返回结构化原因。L1 运行时接收到阻断后会向上报告：遥测推送会包含该阻断事件及其原因。L3 的审计服务会通知 L4（通过遥测采集 API），L4 将审批请求呈现在员工的审批收件箱中。当审批人点击批准时，L4 会调用意图网关 API（契约 2，POST /v1/intents/{id}/approve），向 L3 发出释放阻断的信号。L3 清除闸门后，L1 运行时继续执行。

10. Hooks：跨层强制执行架构

Hooks 是该平台中最重要的架构机制。它们在生命周期的每个关键点提供确定性、可保证的强制执行，并贯穿四个层级。本章将详细说明 hooks 如何产生、如何部署、如何执行，以及如何在整个架构中贯通与协同。



10.1 Hooks 起源：L4 管理控制台

Hooks 最初是企业管理员在 L4 管理控制台中用业务语言定义的策略。例如“法务部门在未经过合规审查前不得对外共享文档”这样的策略，会在策略编辑器中被表示为一条结构化规则，包含

作用域（role:legal）、触发条件（在写入/编辑类工具上的 PostToolUse）、动作（使用 prompt hook 检测是否存在对外共享迹象）以及升级处理（若未获得合规审批则阻止完成）。管理控制台会将这条结构化规则传递给 L3 的策略编译器。

10.2 Hooks 编译：L3 策略编译器

策略编译器把结构化规则翻译为可执行的 hook 配置。对于每条规则，它会生成两类产物：hook 的 JSON 配置（指定事件、匹配器、handler 类型与参数）以及具体的强制执行工件（command handler 对应的 shell 脚本，HTTP handler 对应的端点 URL，prompt handler 对应的提示词模板，或 agent handler 对应的子智能体配置）。编译器会在沙箱化的 L1 会话中用样例事件测试编译后的 hooks。若测试通过，编译结果会被打包为一次 managed-settings.json 更新。

10.3 Hooks 部署：L3 托管设置

策略部署 API（L3/L4 契约 1）接收编译后的 managed-settings.json 更新。L4 的配置存储会记录新版本及其作者、时间戳，以及与上一版本的 diff。L3 以原子方式将更新分发给所有活跃与未来的 L1 实例。活跃会话会在其下一个生命周期事件时接收新 hooks。新会话会在 SessionStart 时接收 hooks。部署是原子的：要么所有实例都收到更新，要么整体回滚。L3 也会存储 hook 的作用域层级：受管 hooks 可在企业、BU、部门、团队层级配置，确保更低层级的策略只会收紧而不会放松更高层级的策略。

10.4 Hooks 桥接：L1 Hooks 桥接器

当 L1 的运行时选择器创建会话时，适配器会把受管 hooks 加载到运行时中。对于 harness 智能体（Octo、Claude Code），hooks 直接加载到运行时的原生 hook 系统里，几乎没有额外开销。对于不原生支持 hooks 的运行时（Aider、Goose、LangGraph 智能体等），L1 的 hook bridge 会在外部进行拦截。bridge 会包装每一次工具调用：框架决定调用某工具 → bridge 拦截 → 运行 PreToolUse hooks → 若允许则转发至工具 → bridge 拦截结果 → 运行 PostToolUse hooks → 若未被阻断则把结果返回给框架。bridge 以 sidecar 容器形式与运行时同部署以降低延迟。bridge 也处理 skill 作用域 hooks：当加载某个 workflow-skill 且其 frontmatter hooks 被激活时，bridge 会将其加入与受管 hooks 并行的拦截管线。

10.5 Hooks 执行：L1 内联触发

在任意 L1 智能体实例的执行过程中，hooks 会在每个生命周期事件上以内联方式触发：

- **SessionStart：**会话开始时触发。受管 hooks 将组织上下文（用户角色、MCP 连接器健康状态、配额状态）注入智能体的初始上下文。配额强制 hooks 会检查剩余配额，若超出则阻止会话创建。

- **UserPromptSubmit：**用户提交消息、进入处理之前触发。kernel boot skill 的模式路由器可作为该事件上的 command handler 实现，把请求分类为 SIMPLE（绕过 kernel）或 KERNEL（启用结构化执行）。其他受管 hooks 也可以注入策略上下文，或阻断被禁止的请求模式。
- **PreToolUse：**主要强制点。在每次工具执行前触发。所有匹配的 hooks 会并行触发（受管、skill 作用域、项目、用户）。若任一 hook 返回 deny（exit code 2），该工具调用会被阻断，拒绝原因会作为上下文反馈给智能体。随后智能体可以自适应调整，选择其他工具，或告知用户。访问控制、安全检查与领域护栏通常在此执行。
- **PostToolUse：**工具成功执行后触发。承担两类职责：质量评估（prompt hooks 评估输出是否存在异常、合规问题，或是否满足 workflow-skill 定义的质量标准）以及审计记录（异步 HTTP hooks 将执行上下文推送到 L3 审计服务而不阻塞执行）。若 PostToolUse hook 阻断，阻断原因会反馈给智能体，智能体必须处理问题后才能继续推进。
- **PostToolUseFailure：**工具调用失败时触发。向智能体上下文注入诊断信息（例如“Google Drive 的 MCP 连接器返回 503，可能暂时不可用”），从而启用 kernel 的自适应恢复：智能体读取诊断后决定重试、跳过该步，或升级处理。
- **PermissionRequest：**智能体请求某个工具权限时触发。受管 hooks 可自动批准安全操作，并将敏感操作升级处理。升级会在 L4 的审批收件箱中创建一条审批请求。
- **Stop：**智能体准备结束响应时触发。作为最终质量闸门：exit code 2 会强制智能体继续工作（修复 hook 识别出的质量问题）。exit code 0 允许完成。Stop hook 也会向 L3 审计服务推送一份最终的遥测摘要。
- **PreCompact：**上下文窗口压缩前触发。允许某个 hook 在详细对话历史被压缩前，将当前 kernel 执行状态摘要保存到 L4 的会话存储中（通过异步 HTTP hook）。这对维持长时运行工作流的执行一致性至关重要。

10.6 Hooks 遥测流：L1 → L3 → L4

每次工具调用后，异步 PostToolUse HTTP hooks 会把遥测从 L1 推送到 L3 的审计服务，这是主要的反馈机制。遥测包含：session ID、runtime ID、用户身份、工具名称、工具输入（已清洗）、工具输出（已截断）、执行耗时、hook 决策（触发了哪些 hooks、各自做了什么决策）以及资源消耗（tokens、计算时长）。L3 审计服务存储这些事件，并通过遥测采集 API（契约 4）使其对 L4 的可观测性枢纽可用。L4 会在仪表盘中对数据进行聚合与可视化，从而形成完整的可观测性管线：L1 产生数据，L3 捕获并存储，L4 展示并分析。

10.7 Hooks 审批流：L1 → L3 → L4 → L1

当某个 hook 以“等待人工审批”为条件阻断执行时，会触发一个跨层流程。该 hook 在 L1 中阻断并返回结构化原因。异步遥测推送会包含一条 blocked_pending_approval 事件。L3 的审

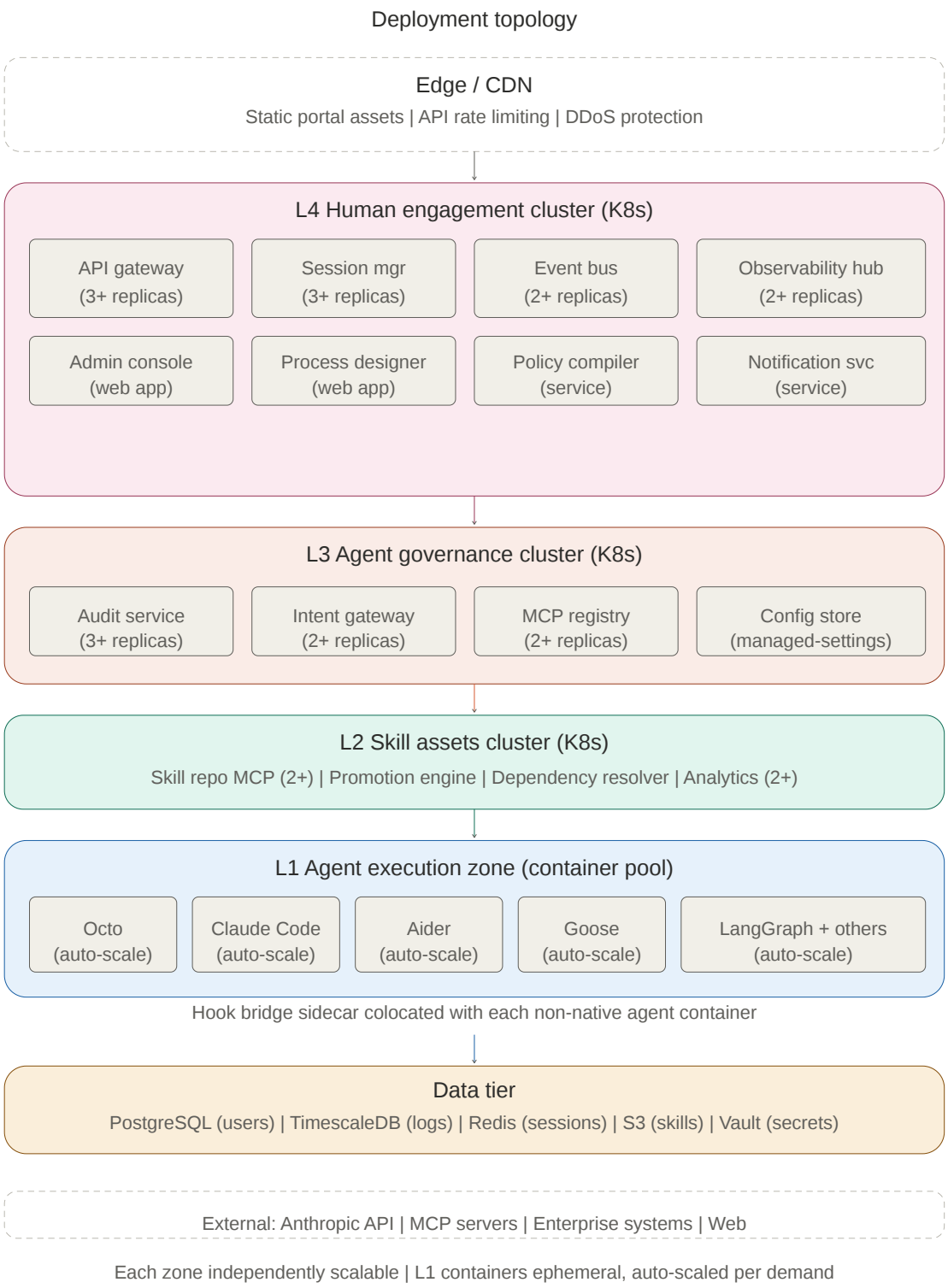
计服务会通过遥测采集 API 通知 L4。L4 的通知系统会在员工的审批收件箱中呈现该请求（或通过 Slack/邮件通知）。审批人审阅上下文后点击批准或拒绝。L4 随后调用意图网关 API（契约 2，POST /v1/intents/{id}/approve）以释放或取消被阻断的工作流。L3 清除闸门，L1 的运行时继续执行。这是唯一一种在执行期间需要跨层往返的 hook 流程，其余 hooks 均以内联方式完成。

10.8 Hooks 保证：拒绝优先（Deny always wins）

其核心保证是：只要任一作用域层级的 hook 拒绝了某个动作，无论其他 hooks 如何决定，也无论智能体推理得出什么结论，该动作都会被阻止。这不是概率性的建议，而是由 hook 执行引擎强制执行的确定性规则。受管 hooks 不能被 skill 作用域 hooks、项目 hooks 或用户 hooks 覆盖。skill 作用域 hooks 不能被项目 hooks 或用户 hooks 覆盖。这种分层的“拒绝优先”模型正是 hooks 适用于企业治理的原因：管理员可以保证策略必然被执行，即便 skill 作者、项目配置或智能体本身更倾向于放行。

11. 部署拓扑

平台部署跨越六个基础设施分区，每个分区都可独立扩展。该架构面向基于 Kubernetes 的容器编排设计，但也可适配无服务器或基于虚拟机的部署方式。四个层级中的每一层都映射到各自的部署分区，并配有共享的边缘与数据层。


Deployment topology
Edge / CDN
Static portal assets | API rate limiting | DDoS protection
L4 Human engagement cluster (K8s)
API gateway (3+ replicas)
Session mgr (3+ replicas)
Event bus (2+ replicas)
Observability hub (2+ replicas)
Admin console (web app)
Process designer (web app)
Policy compiler (service)
Notification svc (service)
L3 Agent governance cluster (K8s)
Audit service (3+ replicas)
Intent gateway (2+ replicas)
MCP registry (2+ replicas)
Config store (managed-settings)
L2 Skill assets cluster (K8s)
Skill repo MCP (2+) | Promotion engine | Dependency resolver | Analytics (2+)
L1 Agent execution zone (container pool)
Octo (auto-scale)
Claude Code (auto-scale)
Aider (auto-scale)
Goose (auto-scale)
LangGraph + others (auto-scale)
Hook bridge sidecar colocated with each non-native agent container
Data tier
PostgreSQL (users) | TimescaleDB (logs) | Redis (sessions) | S3 (skills) | Vault (secrets)
External: Anthropic API | MCP servers | Enterprise systems | Web
Each zone independently scalable | L1 containers ephemeral, auto-scaled per demand

11.1 边缘 / CDN 分区

托管静态门户资源（Web 应用、移动端 PWA）、API 限流与 DDoS 防护。为员工门户提供 HTML/JS/CSS，并将动态 API 调用路由到 L4 服务集群。可使用 CloudFront、Cloudflare 或同等 CDN。

11.2 L4 人机协作集群

在 Kubernetes 命名空间中以可独立伸缩的部署方式承载全部 L4 服务。包括：API 网关（3+ 副本，按请求速率水平自动伸缩）、会话管理器（3+ 副本，按活跃会话数自动伸缩）、事件总线（2+ 副本，底层消息队列可用 SQS、Kafka 或 NATS）、可观测性枢纽（2+ 副本，读密集型负载）、管理控制台与流程设计器（Web 应用，各 2+ 副本）、策略编译器（服务，按策略发布频率伸缩）、通知服务（2+ 副本，集成推送通知与邮件服务商）。所有 L4 服务通过服务网格（Istio 或 Linkerd）在集群内通信，并启用 mTLS。

11.3 L3 智能体治理集群

在 Kubernetes 命名空间中承载 L3 治理服务。包括：审计服务（3+ 副本，写密集型，接收来自所有 L1 实例的异步遥测）、意图网关（2+ 副本，处理事件驱动的工作流分发与审批闸门释放）、MCP 注册表（2+ 副本，为所有运行时提供连接器健康检查）、managed-settings.json 的配置存储（基于 Consul、etcd 或 Kubernetes ConfigMaps，在会话创建时原子分发给 L1 实例）。L3 服务本身无状态；所有持久化状态都在数据层。L3 向上通过五个 L3/L4 API 契约与 L4 通信，向下通过在会话创建时注入 hooks 的方式触达 L1。

11.4 L2 技能资产集群

在 Kubernetes 命名空间中承载 L2 运营服务。包括：Skill 仓库 MCP server（2+ 副本，向所有 L1 智能体提供 skill 读取，并向 L4 流程设计器提供 skill 写入）、晋升引擎（2+ 副本，结合来自 L3 的基于角色闸门管理生命周期迁移）、依赖解析器（1+ 副本，在晋升与读取时校验 skill 兼容性）、访问控制服务（与仓库同部署，对每次请求强制组织范围控制）、版本管理器（与仓库同部署，处理版本与回滚）、使用分析引擎（2+ 副本，处理来自 L3 审计服务的遥测以生成逐 skill 的质量指标）。L2 将 skill 内容存放在对象存储（S3/GCS），元数据存放在 PostgreSQL。

11.5 L1 智能体执行区域

用于承载智能体运行时实例的容器池。每种运行时类型都有独立镜像：Octo（Claude 运行时 + kernel boot skill + 受管 hooks）、Claude Code（Claude 运行时 + 编码工具 + 受管 hooks）、Aider（Python 运行时 + Aider CLI + hook bridge sidecar）、Goose（Rust 运行时 + Goose 二进制 + hook bridge sidecar）、LangGraph 智能体（Python 运行时 + LangGraph + 适配器 + hook bridge sidecar）。L1 容器是临时的：会话开始时创建，会话结束时销毁。按活跃会话数自动伸缩，并为每种运行时配置最小/最大实例数。hook bridge 作为 sidecar 容器与每个非原生运行时同 Pod 部署，共享 Pod 网络以将拦截延迟降到最低。

资源隔离：每个 L1 容器设置独立的 CPU/内存限制，使用隔离文件系统（tmpfs 或临时卷用于 /home/claude），并将网络出站限制为允许访问的 MCP server 端点、L2 skill 仓库与 L3 审计服务端点。任何 L1 容器都不能访问其他 L1 容器或任何 L4 服务；遥测按 L1 → L3（异步 hooks）→ L4（遥测采集 API）流动，从而防止被攻陷的智能体访问持久化状态、用户数据库或管理界面。

11.6 数据层

用于跨层持久化状态的托管数据库服务：

- **PostgreSQL：**用于 L4 用户数据库（画像、角色、配额）、L3 配置存储（策略、RBAC 规则）以及 L2 技能元数据（版本、访问范围、晋升状态、依赖关系）。主库 + 只读副本。支持时间点恢复（PITR）。
- **TimescaleDB 或 ClickHouse：**用于 L4 执行日志与 L3 审计事件存储。针对时序追加写入与分析查询优化。按数据类别配置保留策略。
- **Redis：**用于 L4 会话存储（对话状态、用于跨会话续接的内核状态快照）。集群模式并启用持久化（AOF）以保证耐久性。对遗弃会话采用基于 TTL 的过期清理。
- **对象存储（S3/GCS/Azure Blob）：**用于 L2 技能仓库内容（SKILL.md 文件、模板、资产）、L1 输出文件存储（生成文档、制品）以及备份存储。
- **密钥库（HashiCorp Vault / AWS Secrets Manager）：**用于存放 MCP 连接器凭据（OAuth token、API key）、数据库连接串、部署签名密钥。自动轮换。绝不存入配置文件或环境变量。

11.7 外部集成

Anthropic API（为所有 harness 智能体提供 Claude 模型推理）、MCP 服务器（Gmail、Calendar、Drive 以及企业自定义 MCP 服务器）、企业系统（ERP、CRM、HRIS，既可由 L4 连接器织网访问，也可由 L1 MCP 连接器在执行期间访问）、Web（用于 L1 智能体的搜索与抓取检索能力），以及身份提供方（Azure AD、Okta，用于 SSO/SAML 认证）。

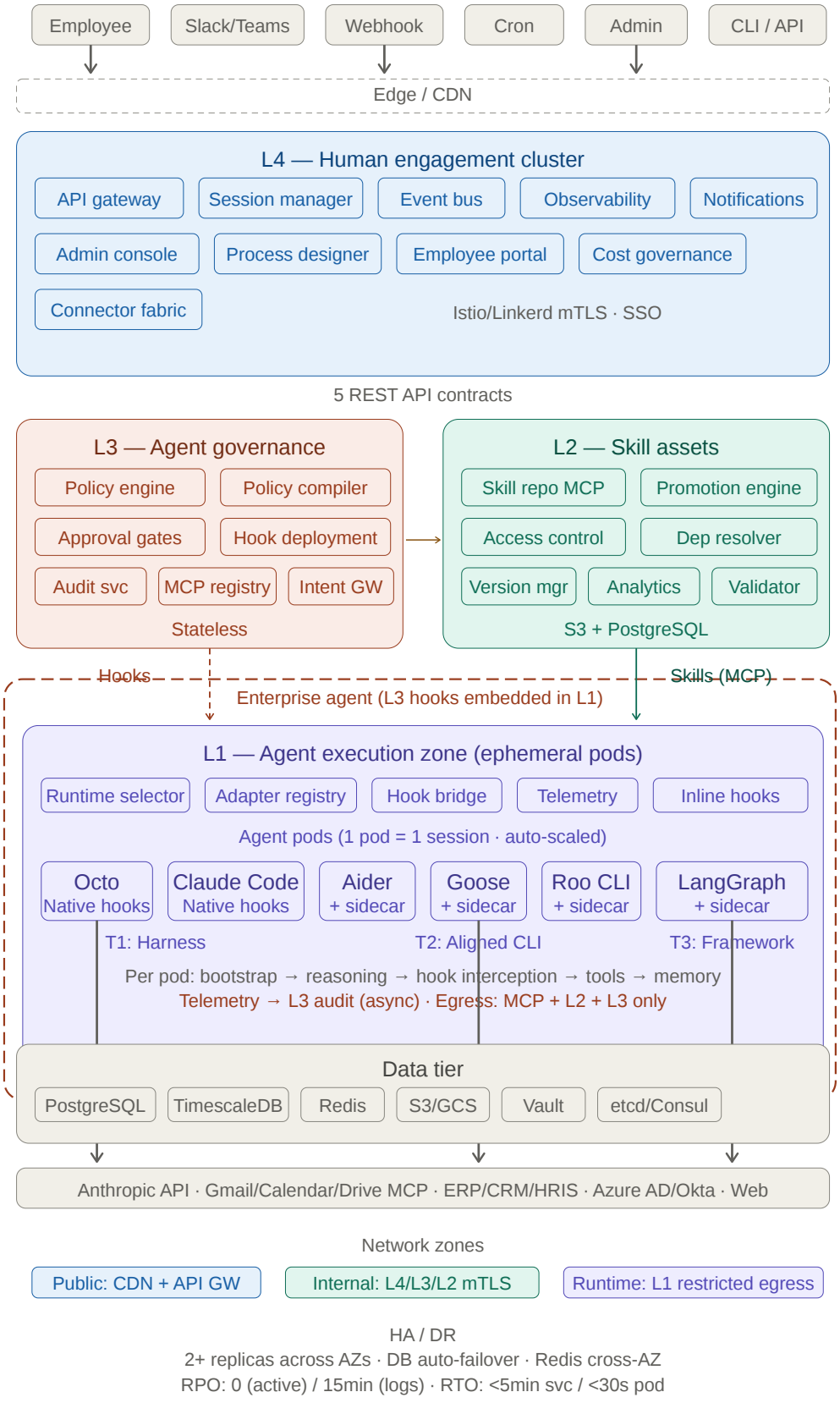
11.8 网络架构

采用三大网络分区并对入站与出站流量进行控制。公网区包含 CDN 与 API 网关的对外入口。内网区承载 L4、L3 与 L2 服务，并通过带 mTLS 的服务网格通信。运行时区承载 L1 容器并限制出站访问：L1 只能访问 MCP 服务器端点、L2 技能仓库以及 L3 审计服务。L1 容器不能直接访问 L4；遥测按 L1 → L3（异步 hooks）→ L4（遥测采集 API）的路径流动，从而避免被攻陷的智能体访问持久化状态、用户数据库或管理界面。

11.9 高可用与容灾恢复

所有 L4、L3 与 L2 服务均以跨可用区的 2+ 副本运行。数据库主从切换自动化。会话存储（Redis 集群）跨可用区复制，以在可用区故障时维持会话连续性。L1 容器无状态且可恢复：当智能体失败时，会以从 L4 会话存储初始化的新实例进行替换。RPO（恢复点目标）：活跃会话为零（实时复制），执行日志为 15 分钟（异步批量写入）。RTO（恢复时间目标）：L4/L3/L2 服务故障切换小于 5 分钟，L1 智能体替换小于 30 秒。

EAASP enterprise deployment: concrete forms and operational relationships



Network zones

Public: CDN + API GW | Internal: L4/L3/L2 mTLS | Runtime: L1 restricted egress

HA / DR

2+ replicas across AZs · DB auto-failover · Redis cross-AZ

RPO: 0 (active) / 15min (logs) · RTO: <5min svc / <30s pod

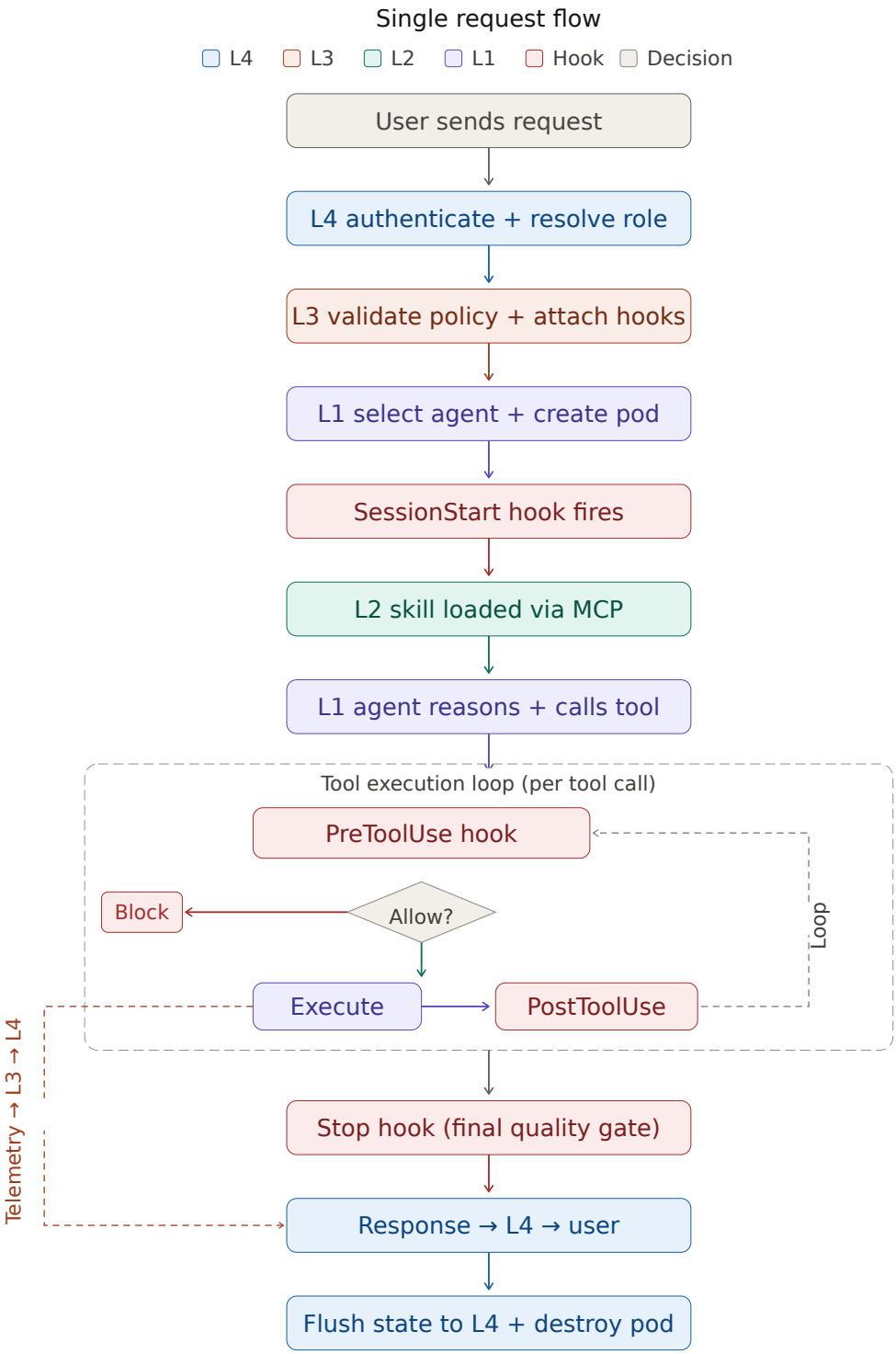
L4 人机协作集群（Human Engagement Cluster）——一个包含 7+ 个微服务的 K8s 命名空间，也是用户真正接触到的平台层。API 网关（FastAPI，3+ pods）是所有访问的单一入口，包括员工门户、Slack/Teams、Webhooks、CLI、管理控制台。会话管理器（支持 WebSocket，3+ pods）是编排"三方握手"的中枢。事件总线（Kafka/NATS，2+ pods）把业务触发（CRM webhook、定时任务、数据库变更事件）转换为结构化意图。四个数据存储（PostgreSQL、TimescaleDB、Redis、S3）都归 L4 所有。

L3 治理集群（Governance Cluster）——一个独立的 K8s 命名空间，包含 7 个无状态服务。关键的运行关系是：当 L4 的会话管理器创建会话时，会调用 L3；L3 解析用户的组织层级（企业 → 事业部 → 部门 → 团队），编译分层的受管 hooks，并将其附加到会话载荷中。hook 部署服务通过 etcd/Consul 以原子方式分发 managed-settings.json。审计服务是写入量最高的服务（接收来自每个 L1 pod 中每一次工具调用的异步遥测）。

L2 技能资产集群（Skill Assets Cluster）——另一个 K8s 命名空间。Skill 仓库 MCP server 是唯一的访问入口：L1 智能体通过 MCP 协议直接连接到它（不经由 L3 路由）。当智能体调用 `skill_read` 时，L2 的访问控制会依据 L3 的 RBAC 校验用户的组织范围，然后返回包含 frontmatter hooks 的 SKILL.md 内容。L3 治理哪些内容可以晋升（基于角色的闸门），但由 L2 直接向 L1 提供内容。

L1 智能体执行区域（Agent Execution Zone）——按会话创建的临时容器 pod，每个会话一个 pod。每个 pod 包含智能体运行时，以及（对非原生运行时）共享 Pod 网络的 hook bridge sidecar。Tier 1 pods（Octo、Claude Code）原生加载 hooks，几乎零额外开销。Tier 2/3 pods（Aider、Goose、LangGraph）由 sidecar 拦截每一次工具调用。Pod 在会话开始时创建、会话结束时销毁，并按活跃会话数自动伸缩。出站网络仅允许访问三个端点：MCP servers、L2 skill 仓库、L3 审计服务。

单次请求的运行流转：
用户打开 Slack → 边缘层路由到 L4 API 网关 → SSO 认证 → 会话管理器启动三方握手 → L3 附加受管 hooks → L1 选择器挑选运行时 → 创建带 hooks 的 pod → 用户消息在 L4↔L1 间流式传输 → 每次工具调用：触发 PreToolUse hooks（包括来自 L2 的 skill 作用域 hooks）→ 执行工具 → 触发 PostToolUse hooks（质量检查 + 异步审计写入 L3）→ 响应流回 → 会话结束 → pod 销毁，状态写回 L4 会话存储。


Single request flow
L4
L3
L2
L1
Hook
Decision
User sends request
L4 authenticate + resolve role
L3 validate policy + attach hooks
L1 select agent + create pod
SessionStart hook fires
L2 skill loaded via MCP
L1 agent reasons + calls tool
Tool execution loop (per tool call)
PreToolUse hook
Block
Allow?
Loop
Execute
PostToolUse
Telemetry → L3 → L4
Stop hook (final quality gate)
Response → L4 → user
Flush state to L4 + destroy pod

12. 演进策略

平台通过五个阶段逐步演进。每个阶段都能独立交付有价值的能力，并在不需要返工的前提下在前一阶段基础上继续构建。各阶段与四层架构对齐：阶段 1 建立 L1 基础，阶段 2 构建 L2 与 L1 内部抽象，阶段 3 搭建 L3 与 L4 基础能力，阶段 4 完成各层并走向成熟，阶段 5 打开生态。

阶段 1：L1 内核 + 基础 hooks（第 1–4 周）

为 Octo 创建 kernel-boot skill，包含模式路由器、意图解析器、资源协调器与执行循环。将模式路由器实现为 UserPromptSubmit hook（command handler）。增加用于上下文注入的 SessionStart hook。增加用于遥测采集的 Stop hook。部署到试点人群。验证：简单任务不受影响；复杂任务获得结构化执行。为常见的多步任务创建 3–5 个初始 workflow-skills。

交付物：Octo 中 kernel boot skill 可用，三个 hook 事件生效（SessionStart、UserPromptSubmit、Stop），3–5 个 workflow-skills 在试点用户中验证通过。无需基础设施变更——所有能力均运行在现有 Octo 运行时之内。

阶段 2：L2 技能资产 + L1 抽象（第 5–12 周）

构建 L2 技能资产层：skill 仓库 MCP server、晋升引擎（draft → tested → reviewed → production 流水线）、访问控制服务与版本管理器。将基于文件系统的 skills 迁移到 L2。构建 MCP 注册表作为配套的 MCP server。在 L1 内构建内部抽象机制：运行时选择器（任务匹配与盲盒策略）、适配器注册表（为 Octo 与 Claude Code 这两类 Tier 1 harness 智能体提供薄适配器）以及遥测采集器。启用 Octo 与 Claude Code 的盲盒对比，以生成经验驱动的质量数据。

交付物：L2 上线并支持版本化 skills 与受治理的晋升流水线；L1 抽象可在两个 Tier 1 运行时之间进行选择；盲盒质量评分启用；5–10 个带作用域 hooks 的 workflow-skills 在 production 中可用。

阶段 3：L3 智能体治理 + L4 基础能力（第 13–20 周）

构建 L3 智能体治理层：策略引擎（将管理员规则编译为受管 hooks）、审计服务（接收来自 L1 的遥测）、审批闸门（通过 PreToolUse 与 PostToolUse 阻断，并与 L4 审批收件箱集成）以及 managed-settings.json 的部署流水线。构建 L4 人机协作层基础能力：会话管理器（L1 生命周期与路由）、API 网关（鉴权与限流）、带策略编译器的管理控制台，以及员工门户（Web + Slack/Teams 内嵌）。部署五个 L3/L4 API 契约。在 L1 内构建面向非原生运行时的 hook bridge，并将 Aider 与 Goose 作为 Tier 2 对齐型智能体加入运行时池。启用在三类智能体间的任务匹配运行时选择。

交付物：L3 治理上线并以受管 hooks 方式在所有运行时上强制执行；L4 通过门户与管理控制台服务员工；hook bridge 在 Tier 2 智能体上验证通过；五个 API 契约上线；运行时池包含四个智能体（Octo、Claude Code、Aider、Goose）。

阶段 4：完整 L4 + L3 成熟 + Tier 3（第 21–30 周）

完成 L4：用于自动化（无人工交互）工作流的事件总线，支持定时任务、Webhook 与 CDC 触发；可观测性枢纽，提供运行时对比仪表盘与成本追踪；流程设计器，提供可视化 skill 构建器与仿真模式；通知服务，用于主动告警与审批收件箱路由。推动 L3 成熟：通过 SessionStart hook 实现配额强制与成本治理，增加预算告警与按组织单元的成本分摊报表。扩展 L2：加入依赖解析器与使用分析引擎，将质量指标反馈到 L4 可观测性。扩展 L1：加入基于 LangGraph 的运行时以支持长时运行、可崩溃恢复的工作流（Tier 3 框架型智能体），提供厚适配器与完整的 skill → graph 翻译。

交付物：四层全部上线并稳定运行；自动化工作流可无界面执行；成本治理生效；运行时池覆盖三个 tier 且包含 5+ 个智能体；可观测性支持跨运行时质量对比。

阶段 5：生态扩展（持续进行）

开放平台 API，以支持外部系统集成与第三方智能体接入。扩展 L1 运行时池：加入 Roo Code CLI、Cline CLI（Tier 2），以及 CrewAI、Pydantic AI（Tier 3）。构建运行时认证流水线：自动化接口契约测试、盲盒质量基准测试与安全扫描。为 L4 增加多租户能力，完整建模组织层级（企业 → 事业部 → 部门 → 团队）。扩展 L2 以支持技能市场能力：跨组织的 skill 共享与访问控制。为平台 API 的使用方构建 SDK。持续评估新出现的智能体与框架以纳入运行时池。

交付物：开放生态与认证流水线落地；运行时池包含 8+ 个智能体；支持多租户部署；技能市场可用；发布平台 SDK。

13. 设计反模式

- **在需要 hooks 的地方用 prompts：**如果某件事必须每次都发生（策略、校验、审计），就必须用 hook。Prompt 只是建议，hook 才是保证。
- **过度使用高成本 hooks：**并非每个检查都需要 prompt hook 或 agent hook。先从 command hook（毫秒级）开始，只有在需要语义评估时才升级到 prompt hook（秒级）。
- **跳过模式路由器：**把所有请求都路由进内核会给简单任务增加时延。应将 80% 的请求归类为 SIMPLE，以实现零内核开销。
- **没有受管 hooks 的治理：**基于推理的策略强制可能被智能体忽略。只有受管 hooks 才能提供“拒绝优先（deny-always-wins）”的保证。
- **单体化的 hook 脚本：**保持 hooks 小、快、单一职责。它们并行运行，一个慢 hook 会阻塞所有事情。应拆分为多个独立 hooks。
- **没有作用域钩子的技能：**仅依赖自然语言说明来做质量强制的 skill，最终会漏掉标准。为不可妥协的检查增加钩子。

- **绕过 skill 仓库：**从文件系统加载 skills 会丢失版本管理、访问范围控制与使用分析。所有 skills 都应通过仓库 MCP server 获取。
- **把文件系统当作持久化：**L1 的文件系统在会话间会重置。任何需要跨会话保留的内容，都应使用 L2 hooks（遥测）或 L3 MCP（会话存储）。
- **忽视运行时的无界面能力：**VS Code 扩展（完整 Cline、完整 Roo Code）不能作为服务端池运行时。只有 CLI 变体符合要求。扩展可以作为客户端运行时，并由本地受管 hooks 治理。
- **双重抽象：**如果 skill 系统已经处理了某个模式，就不要再在 L2 包一层抽象。L2 调用 skills，而不是重实现它们的逻辑。

14. 完整设计决策轨迹

该表追溯了五次设计迭代中每一个关键架构问题。

问题	v1–v2	v3（hooks）	v4（运行时池）	v5 最终版
L1 是什么？	仅 Octo	Octo + 内核 + hooks	L1 内部抽象 + 运行时池	具备内部抽象机制的 L1 智能体运行时池；任何实现接口契约的智能体都可加入；3 个运行时 tier。
工作流怎么做？	YAML → skills	Skills + hooks	同上 + 亲和性（affinity）	带作用域 hooks + 运行时亲和性的 workflow-skills；智能性与确定性保障合一。
治理怎么做？	未定义 → policy	受管 hooks（managed hooks）	• hook bridge	对所有运行时统一生效的受管 hooks + hook bridge；策略从 L3 管理控制台编译；拒绝优先（Deny always wins）。
L3 是什么？	未涉及	未涉及	4 个平面，5 个 API	体验 + 集成 + 控制 + 持久化四平面；5 个 L3/L4 契约；多租户；安全架构。
运行时如何选择？	N/A（单一）	N/A	5 类策略	任务匹配、盲盒对比、用户偏好、A/B 测试、成本优化；能力清单（capability manifests）；运行时生命周期管理。

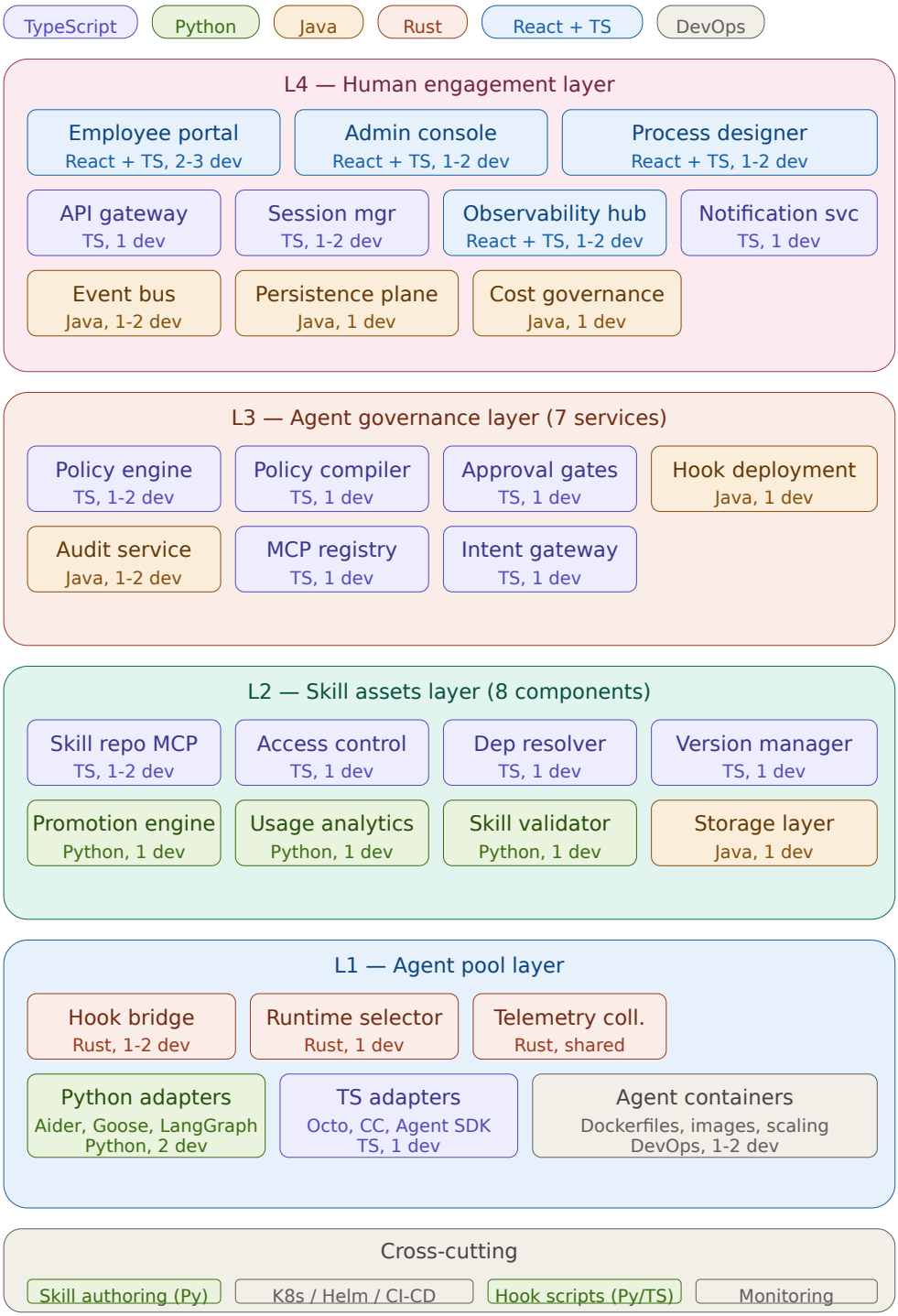
选哪些智能体？	仅 Octo	仅 Octo	映射 4 个 tier	T1：harness（Octo、Claude Code、SDK）；T2：对齐型 headless（Aider、Goose、Roo CLI、Cline CLI）；T3：框架型；T4：暂缓。
状态模型？	未定义	短暂态（ephemeral）	L3/L2/L1 分层拆分	L3 持久化（DB）；L2 配置（managed-settings）；L1 短暂态（会话内）；通过 hooks 或 MCP 跨会话存活。
分几层？	两层（L2+L1）	三层 + hooks	四层 + hooks	四层（L4、L3、L2、L1）+ 3 项跨层机制（hooks、skills、MCP）。
无界面（headless）检查？	N/A	N/A	未检查	VS Code 智能体（Cline、Roo）只能使用 CLI 变体；扩展为客户端形态；Aider/Goose 原生 headless。
演进路径？	未涉及	未涉及	扩展，5 个阶段	扩展既有 Octo：5 阶段（内核 → 仓库 → L3 + T2 → 完整 L3 + T3 → 生态）；每个阶段都可独立交付价值。

企业自主智能体支撑平台（EAASP）对运行时无关、治理统一、以技能承载智能，并且可渐进式演进。L4 服务人类用户。L3 治理智能体。L2 管理知识。L1 执行工作。Hooks 贯穿一切。

15. 实施团队分工

本章基于四种可用技术栈将平台的各个组件映射到对应的开发团队：TypeScript（前端使用 React）、Python、Java，以及 Rust（小团队）。分工遵循两项原则：每个组件应使用最符合其技术需求的语言来实现，并且每位开发者应尽量只在单一层级内工作，以保持架构边界的纪律性。

Team assignment by layer and language



Team summary

TypeScript/React: 10-12 dev (L4 frontend + L3 governance + L2 repo/access/resolver/version)
Java: 6-7 dev (L4 infra + L3 audit/deploy + L2 storage) | Python: 6-7 dev (adapters, analytics, skills)
Rust: 2-3 dev (hook bridge, selector, telemetry) | DevOps: 1-2 dev | Total: 25-31 dev

15.1 TypeScript / React 团队（10–12 名开发者）

这是人数最多的团队，负责两类工作。

前端子团队（React + TypeScript）构建所有 L4 面向用户的应用：员工门户（Web + 移动端 PWA + Slack/Teams 内嵌）、管理控制台（策略编辑器、角色管理、运行时池管理）、流程设计器（带仿真模式的可视化 Skill 构建器）、以及可观测性枢纽仪表盘。

后端子团队（Node.js + TypeScript）构建 L4 的异步 I/O 服务（API 网关、会话管理器、通知服务）以及 L3 治理层的核心服务：策略引擎（RBAC 规则管理、条件策略评估）、策略编译器（将规则编译为 hook JSON，并对配置 schema 做强类型约束，支持沙箱测试）、审批闸门（阻断与释放管理、L4 收件箱通知）、意图网关（工作流分发）、以及 MCP 注册表。

TypeScript 是实现 L3 治理逻辑的自然选择，因为 hook 配置是复杂的 JSON schema，类型安全可以有效防止误配置。

TypeScript 也是实现 MCP server 的主流语言，因此也适合作为 L2 核心服务的主要实现语言：Skill 仓库 MCP server（7 个工具，是平台访问最频繁的服务）、访问控制服务（对每次 Skill 读写按组织范围校验）、依赖解析器（图遍历以保证跨 Skill 的兼容性）、以及版本管理器（语义化版本、回滚、diff 跟踪）。

L2 的 TypeScript 服务构成一个紧密协作的服务组，复用同一套 PostgreSQL 元数据 schema 与对象存储客户端。

15.2 Java 团队（6–7 名开发者）

负责企业级“骨干”组件，Java 在可靠性、事务管理与企业集成工具链方面的成熟度最具优势。

在 L4：事件总线（Kafka/NATS 集成、定时/Webhook/CDC 触发、Exactly-once 投递）与持久化平面（PostgreSQL 连接池、TimescaleDB 写入、Redis 会话存储）。

在 L3：审计服务（从所有 L1 实例高写入量接入遥测，必须在持续吞吐下不丢数据）与 hook 部署服务（managed-settings.json 通过 etcd/Consul 原子分发到所有 L1 实例，支持版本追踪、回滚与差异管理）。

在 L2：存储层（S3/GCS 集成以存放 Skill 内容、PostgreSQL 元数据 schema、对高频访问的 production skills 做 read-through 缓存）。

Java 的 JDBC 生态、久经考验的连接池与成熟的可观测性工具（Micrometer、Prometheus exporters），使其非常适合实现“绝不能丢数据”的关键组件。

15.3 Python 团队（6–7 名开发者）

负责 AI 相关组件与智能体适配器层。

在 L2：晋升引擎（通过 skill-evaluator 元 Skill 集成基于 ML 的 Skill 评估，并强制执行四阶段生命周期闸门）、使用分析引擎（遥测处理、逐 Skill 质量指标、失败模式分析，使用

pandas/numpy 识别废弃候选）、以及 Skill 校验器（校验 YAML frontmatter 语法、检查 hook 声明是否合法、核对运行时亲和性是否匹配已注册运行时、在接受草稿前验证依赖引用）。

在 L1：面向 Python 生态智能体的适配器（Aider、Goose、LangGraph、CrewAI、Pydantic AI 等均为 Python 原生）。

跨层工作包括：Skill 编写工具（skill-creator、skill-evaluator 元 Skill，基于 LLM API 调用）与 hook handler 脚本（用 Python/shell 实现的 command hooks）。

Python 是智能体生态的主要语言，大量社区智能体与 AI 框架都基于 Python，因此适配器开发者必须同时熟悉平台接口契约与智能体内部 API。

15.4 Rust 团队（2–3 名开发者）

人数最少，聚焦性能最关键的 L1 内部组件。

hook bridge sidecar 是 L1 中最重要的组件，它会拦截每一个非原生智能体的每一次工具调用，并必须以近乎零额外时延强制执行“拒绝优先（deny-always-wins）”。

运行时选择器位于每次会话创建的热路径上，需要将任务画像与能力清单（capability manifests）进行匹配。

遥测采集器与其同置在同一个 sidecar 二进制中，用于归一化高吞吐事件流。

三者共享一个以 sidecar 容器方式部署的 Rust 二进制。

Rust 的零成本抽象、无需 GC 的内存安全与可预测时延，对这些性能直接影响每个用户会话的组件至关重要。

15.5 DevOps 团队（1–2 名开发者）

负责容器镜像（为每种智能体类型编写 Dockerfile，并包含 hook bridge sidecar）、Kubernetes 部署（四层集群的 Helm charts）、CI/CD 流水线（自动化测试、安全扫描、自动化发布）、监控基础设施（Prometheus、Grafana、告警），以及运行时认证流水线（对新加入池的智能体进行自动化接口契约测试）。DevOps 跨越所有层级工作，但不负责应用逻辑。

15.6 分工原则

- **一名开发者，一个层级：**开发者应主要在单一层级内工作，以保持清晰的架构边界。负责 L3 审计服务的 Java 开发者不应同时构建 L4 持久化平面组件，即便两者都使用 Java。跨层工作会引入耦合。
- **适配器语言与智能体一致：**每个智能体适配器必须使用该智能体的原生语言编写。Aider 适配器使用 Python，因为 Aider 是 Python；Claude Code 适配器使用 TypeScript，因为 Claude Code 是 TypeScript。适配器开发者必须深入理解该智能体的内部 API。

- **Rust 仅用于热路径：**不要将 Rust 的使用扩展到 L1 的三个内部组件之外。Rust 团队规模小，组件边界紧凑。其他服务（即便是对性能敏感的审计服务）也更适合使用 Java 的生态成熟度与更大的可用团队规模。
- **按阶段对齐人员投入：**在阶段 1（第 1–4 周）仅需要 Python 与 TypeScript 开发者（kernel boot skill + 初始 hooks）。Rust 开发者在阶段 2 加入（hook bridge）。Java 开发者在阶段 3 加入（事件总线、审计服务）。前端开发者在阶段 3 加入（门户、管理控制台）。从阶段 3 开始需要全员投入。

15.7 人员配置汇总

TypeScript / React	10–12	L4（前端 + 后端）、L3（治理核心）、L2（仓库、访问控制、依赖解析、版本管理）	员工门户、管理控制台、流程设计器、可观测性仪表盘、API 网关、会话管理器、策略引擎、策略编译器、审批闸门、MCP 注册表、意图网关、Skill 仓库 MCP
Java	6–7	L4（基础设施）、L3（审计、部署）、L2（存储）	事件总线、持久化、成本治理、审计服务、hook 部署、L2 存储层
Python	6–7	L2（晋升、分析、校验器）、L1（适配器）、跨层能力	晋升引擎、使用分析、Skill 校验器、Python 适配器（Aider/Goose/LangGraph）、Skill 编写工具
Rust	2–3	L1（内部抽象）	Hook bridge sidecar、运行时选择器、遥测采集器（单一二进制）
DevOps	1–2	全层（基础设施）	容器镜像、K8s/Helm、CI/CD、监控、认证流水线
Total	25–31		

16. 实施团队分工（Python为主）

本章基于一个以 Python 为主的 10–16 人团队，将平台的每个组件映射到具体开发者。团队具备四类语言能力：Python（占多数）、TypeScript + React（前端）、Java（企业级基础设施）、Rust（小团队负责的性能关键模块）。每位开发者负责 2–4 个相关模块，这些模块共享相同的领域逻辑、数据模型或部署单元。不设置只负责单一模块的分工。最小可行团队为 10 人（Py1–5、TS1–2、Java1、Rust1、DevOps1）；随着智能体类型拓展与数据管道吞吐增长，可选角色（Py6、Java2）将团队规模扩展到 16 人。

Team assignment — consolidated roles (10-16 developers)

React + TS | TypeScript | Python | Java | Rust | DevOps

L4 — Human engagement layer

TS/React Dev A — Portal + observability
Employee portal (web, mobile, Slack/Teams), observability hub dashboards

TS/React Dev B — Admin + designer
Admin console (policy rules UI), process designer (visual skill builder)

TS Dev C — L4 backend services
API gateway, session manager, notification service

Java Dev A — L4 infrastructure
Event bus (Kafka/NATS), persistence plane (PG/Redis), cost governance

L3 — Agent governance layer (7 services)

TS Dev D — Governance core
Policy engine, policy compiler, approval gates, MCP registry, intent gateway (same domain logic)

Java Dev B — Audit + deployment
Audit service (high-write telemetry), hook deployment service (atomic managed-settings dist)

L2 — Skill assets layer (8 components)

TS Dev E — L2 MCP
Skill repo MCP (7 tools), access control, dep resolver, version mgr

Py Dev A — L2 lifecycle
Promotion engine, usage analytics, skill validator

Java Dev B (+L3)
Storage layer (S3/PG/cache) Shared with L3 above

L1 — Agent execution layer

Rust Dev A — L1 sidecar binary
Hook bridge + runtime selector + telemetry collector (single binary)

Py Dev B — L1 Python adapters
Aider, Goose, LangGraph adapters, hook handler scripts

Py Dev C — L1 framework adapters
CrewAI, Pydantic AI adapters, skill authoring tools (meta skills)

TS Dev C — L1 TS adapters (+L4)
Octo, Claude Code, Agent SDK adapters (shared with L4 backend)

Cross-cutting + infrastructure

DevOps — containers, K8s, CI/CD, monitoring | Hook scripts (Py/TS) | Cert pipeline

Staffing summary — 10 core + up to 6 expansion

Core team (10 developers, phases 1-3):

TS/React ×5: A portal | B admin+designer | C L4 backend + L1 TS adapters
D L3 governance | E L2 MCP + metadata

Java ×2: A L4 infra | B L3 audit/deploy + L2 storage

Python ×2: A L2 lifecycle | B L1 Python adapters

Rust ×1: A L1 sidecar (hook bridge + selector + telemetry)

Expansion (phases 4-5): +Py Dev C (framework adapters) +Rust Dev B (perf) +DevOps (dedicated infra) +TS/React (scale frontend) = up to 16 total

16.1 Python 团队（5–8 名开发者）

规模最大的团队，负责 L4、L3、L2 与 L1 中绝大多数后端服务。核心团队需要 5 名 Python 开发者（Py1–5）；随着智能体多样性与运营规模增长，最多可增加 3 名开发者（Py6–8）。Python 生态覆盖全部需求：FastAPI 用于高性能异步服务；Python MCP SDK 用于实现 MCP server；SQLAlchemy 用于元数据管理；pandas/numpy 用于分析；并且天然熟悉 AI 智能体生态。

Py Dev 1 — L4 后端 + L1 harness

负责 L4 的异步 I/O 层：API 网关（FastAPI，中间件处理认证、限流、路由）、会话管理器（WebSocket 流式传输、会话生命周期、三方握手协调）、通知服务（推送通知、邮件集成、审批收件箱路由），以及连接器织网（MCP 连接器生命周期、凭据委派到密钥库）。同时负责 L1 中 Octo、Claude Code 与 Agent SDK 的薄 harness 智能体适配器，因为这些适配器是对运行时接口契约的轻量 Python 垫片，并与会话管理器共享连接模型。

Python Dev 2 — L3 治理核心

负责全部五个不依赖 TypeScript 的 L3 治理服务：策略引擎（RBAC 规则管理、条件策略评估、组织层级作用域）、策略编译器（规则→hook JSON 编译、强制执行脚本生成、在隔离的 L1 会话中进行沙箱测试）、审批闸门（阻断/释放状态机、L4 收件箱通知）、MCP 注册表（以 MCP server 形式暴露连接器健康状态与注册能力）、意图网关（事件驱动的工作流分发与路由）。这五个服务共享同一套领域模型——策略、hooks、组织单元——因此非常适合作为单人负责的开发范围。使用 FastAPI + Python MCP SDK 实现。

Py Dev 3 — L2 仓库与访问控制

负责 L2 核心数据服务：技能仓库 MCP 服务器（提供 7 个 MCP 工具：搜索、读取、提交、晋升、列出版本、依赖、使用情况），访问控制服务（基于 L3 的 RBAC 按组织范围校验每一次读写），依赖解析器（用于技能间兼容性的图遍历、在晋升或弃用时检测破坏性变更），版本管理器（语义化版本、回滚、差异跟踪、弃用告警），以及存储层（SKILL.md 内容对接 S3/GCS，元数据使用 PostgreSQL，生产技能采用读穿缓存）。这些服务共享同一套数据库 Schema 与对象存储客户端，因此可以作为一个内聚的职责域统一归属。

Py Dev 4 — L2 生命周期与质量

负责 L2 智能服务：晋升引擎（四阶段生命周期流水线，带基于角色的门禁，并集成评测框架），使用分析引擎（处理来自 L3 审计服务的遥测数据，生成按技能维度的质量指标、失败模式分析、弃用候选识别），技能校验器（YAML frontmatter 语法校验、Hook 声明与已知处理器类型的匹配校验、运行时亲和性与已注册运行时的校验），以及技能编写工具（skill-creator 与 skill-evaluator 两个元技能，基于 LLM API 调用）。这些服务构成技能质量的反馈闭环：执行数据进入分析，分析产出质量分，质量分指导晋升决策与技能改进。

Py Dev 5 — L1 适配器（核心）

负责 L1 中主要的 Python 端智能体适配器：Aider（git 原生，执行循环简单，便于与 Hook Bridge 集成），Goose（MCP 原生，执行模型透明），LangGraph（较厚的适配层，将技能映射为图节点，并支持带检查点的持久化执行）。每个适配器都必须通过智能体的原生 Python API 实现 12 方法的运行时接口契约。该开发者还会编写平台通用的 Python Hook 处理脚本（命令 Hook、HTTP 端点处理器）。如果团队规模最小（没有 Py Dev 6），该开发者还需要以更窄的初始范围（Phase 2 接入的智能体更少）承担 CrewAI 与 Pydantic AI 的适配工作。

Py Dev 6 — L1 适配器（扩展，可选）

负责其余智能体适配器：CrewAI（基于角色的多智能体，将技能转换为 crew），Pydantic AI（类型安全、模型清晰）。同时负责将未来的新智能体接入到适配器池中：编写新适配器、通过运行时认证流水线进行验证，并扩展与 Hook Bridge 的兼容性。该角色在上线时可选：在更小团队下 Py Dev 5 可以覆盖全部 5 个适配器，但拆分该角色可避免随着 Phase 2 之后智能体多样性提升而导致适配器研发速度被单点瓶颈限制。

16.2 TypeScript / React 团队（2 名开发者）

专注于 L4 前端应用。两位开发者使用 React + TypeScript 构建面向用户的交互界面，这些界面由 Python 后端提供服务。

TS/React Dev 1 — Portal（员工门户）

负责员工门户：Web 应用、移动端 PWA、Slack/Teams 内嵌集成。处理多渠道路由、跨设备的会话连续性，以及会话体验（Session UX）。员工门户是员工与平台交互的主要入口。

TS/React Dev 2 — Admin and operations（管理与运维）

负责管理控制台（策略规则编辑器、角色管理、智能体运行时池管理）、流程设计器（可视化技能搭建，支持拖拽与仿真模式）、以及可观测性中枢仪表盘（运行时对比、成本跟踪、工作流执行链路追踪）。这三套应用共享同一套管理端设计系统与 API 客户端库。

16.3 Java 团队（1–2 名开发者）

负责企业级基础设施组件。Java 在可靠性、事务管理与高吞吐数据处理方面的成熟度在这些场景中具有不可替代的价值。

Java Dev 1 — L4 基础设施

负责 L4 企业主干：事件总线（Kafka/NATS 集成、定时任务/ webhook / CDC 触发、Exactly-once 交付保障）、持久化平面（PostgreSQL 连接池、TimescaleDB 时序写入、Redis 会话存储并启用 AOF 持久化）、以及成本治理服务（配额限制、预算跟踪、分摊计费报表）。这三个服务共享 JDBC 连接池与同一数据层。

Java Dev 2 — L3 数据管道（可选）

负责 L3 高写入服务：审计服务（遥测写入入口，接收来自所有 L1 实例的异步 PostToolUse 事件，使用 TimescaleDB 的结构化事件存储）与 Hook 部署服务（通过 etcd/Consul 原子分发 managed-settings.json，版本跟踪、回滚、差异管理）。若团队只有 1 名 Java 开发者，这部分可由 Java Dev 1 吸收，因为审计服务与持久化平面共享同一套 TimescaleDB 基础设施。另一种选择是由 Python（Py Dev 2）使用异步框架实现审计服务，而 Java Dev 1 扩展覆盖 Hook 部署。

16.4 Rust 开发者（1 名）

负责 L1 侧车二进制：Hook Bridge（拦截所有非原生智能体的工具调用，执行 deny-always-wins）、运行时选择器（在每次会话创建时将任务画像与能力清单匹配）、以及遥测采集器（在推送至 L3 审计前对高吞吐事件流做归一化）。这三个组件编译为一个 Rust 二进制，以 sidecar 容器形式与每个非原生智能体同机部署。这是平台最关键的性能热路径代码：Hook Bridge 的延迟会直接影响所有被桥接智能体的每一次工具调用。由于三者高度耦合、共享同一套异步运行时（tokio）并作为单一部署单元上线，一名资深 Rust 开发者可以覆盖该范围。

16.5 DevOps（1 名开发者）

负责全部基础设施自动化：容器镜像（为每种智能体类型编写 Dockerfile，并包含 Hook Bridge sidecar）、Kubernetes 部署（为 L4/L3/L2 服务集群与 L1 容器池编写 Helm charts）、CI/CD 流水线（自动化测试、安全扫描、部署自动化）、监控基础设施（Prometheus、Grafana、告警）、以及运行时认证流水线（对加入智能体池的新智能体进行自动化接口契约测试）。DevOps 跨层协作，但不负责应用业务逻辑。

16.6 分配原则

- **按领域聚类的所有权：**每位开发者负责共享同一领域模型、数据库 Schema 或部署单元的模块。Py Dev 2 负责全部 L3 治理服务，因为它们共享 policy/hook 领域模型。Py Dev 3 负责全部 L2 数据服务，因为它们共享同一套 PostgreSQL schema 与 S3 客户端。Rust 开发者负责整个 sidecar 二进制。任何开发者都不负责彼此无关的模块。
- **Python 承担后端广度：**在 Python 为主的团队配置下，Python 替代 TypeScript 承担所有后端服务，包括 MCP 服务器（通过 Python MCP SDK）、API 服务（通过 FastAPI）与治理逻辑。TypeScript 仅保留给 React 前端。这最大化代码共享并减少语言边界。
- **Java 仅用于数据可靠性：**Java 仅限用于其事务管理、连接池与 Exactly-once 交付保障真正不可替代的组件：事件总线（Kafka）、持久化平面（JDBC）与高写入审计接入。不要将 Java 范围扩展到应用业务逻辑。
- **Rust 仅用于热路径：**Rust sidecar 二进制是唯一 Rust 组件。不要将 Rust 使用扩展到 L1 内部之外。由于三个组件属于同一部署单元，一名 Rust 开发者足够覆盖。

- **按阶段对齐的上手路径：**Phase 1（第 1–4 周）：Py Dev 1 + Py Dev 4（kernel boot skill、初始 hooks、workflow-skills）。Phase 2（第 5–12 周）：加入 Py Dev 3 + Py Dev 5 + Rust Dev 1（L2 仓库、首批适配器、hook bridge）。Phase 3（第 13–20 周）：加入 Py Dev 2 + TS Dev 1–2 + Java Dev 1（L3 治理、门户、基础设施）。Phase 4+：按工作量加入可选角色（Py Dev 6 扩展适配器广度，Java Dev 2 将数据管道隔离）

16.7 人员配置汇总

团队	规模	开发者角色	负责的关键模块
Python	5–8	Py1：L4 后端 + L1 Harness 适配器；Py2：L3 治理（5 个服务）；Py3：L2 仓库 + 访问控制（5 个服务）；Py4：L2 生命周期 + 质量（4 个服务）；Py5：L1 适配器（核心）；Py6：L1 适配器（扩展，可选）	API 网关、会话管理、通知、策略引擎、编译器、审批门禁、MCP 注册中心、意图网关、技能仓库 MCP、访问控制、依赖解析器、版本管理器、存储、晋升引擎、分析引擎、校验器、适配器（Aider/Goose/LangGraph/CrewAI/Pydantic AI）
TypeScript / React	2	Dev1：Portal（员工门户）；Dev2：Admin + 设计器 + 可观测性	员工门户（Web、移动端、Slack/Teams 内嵌）、管理控制台、流程设计器、可观测性仪表盘
Java	1–2	Dev1：L4 基础设施（事件总线、持久化、成本）；Dev2：L3 审计 + Hook 部署（如有）	事件总线、持久化平面、成本治理、审计服务、Hook 部署服务
Rust	1	Dev1：L1 sidecar 二进制	Hook Bridge、运行时选择器、遥测采集器（单一二进制）
DevOps	1	容器、K8s、CI/CD、监控	Dockerfile、Helm Charts、流水线、Prometheus/Grafana、认证流水线
合计	10–16		